

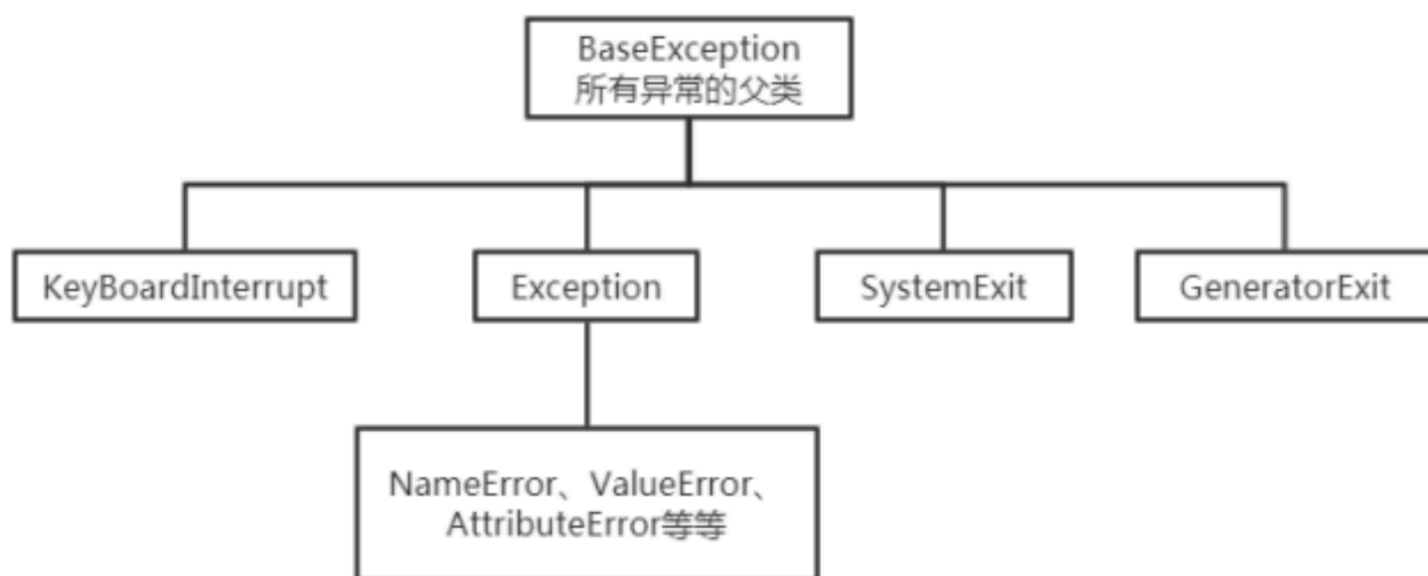
02 python 深入提高

1. 异常

1.1 异常的本质

本质是：当程序出现异常，程序安全的退出、处理完后继续执行的机制。

python中，引进了很多用来描述和处理异常的类，称为异常类。异常类定义中包含了该类异常的信息和对异常进行处理的方法。



python中一切都是对象，异常也采用对象的方式来处理。

处理过程：

1. 抛出异常：在执行一个方法时，如果发生异常，则这个方法生成代表该异常的一个对象，停止当前执行路径，并把异常对象提交给解释器。
2. 捕获异常：解释器得到该异常后，寻找相应的代码来处理该异常。

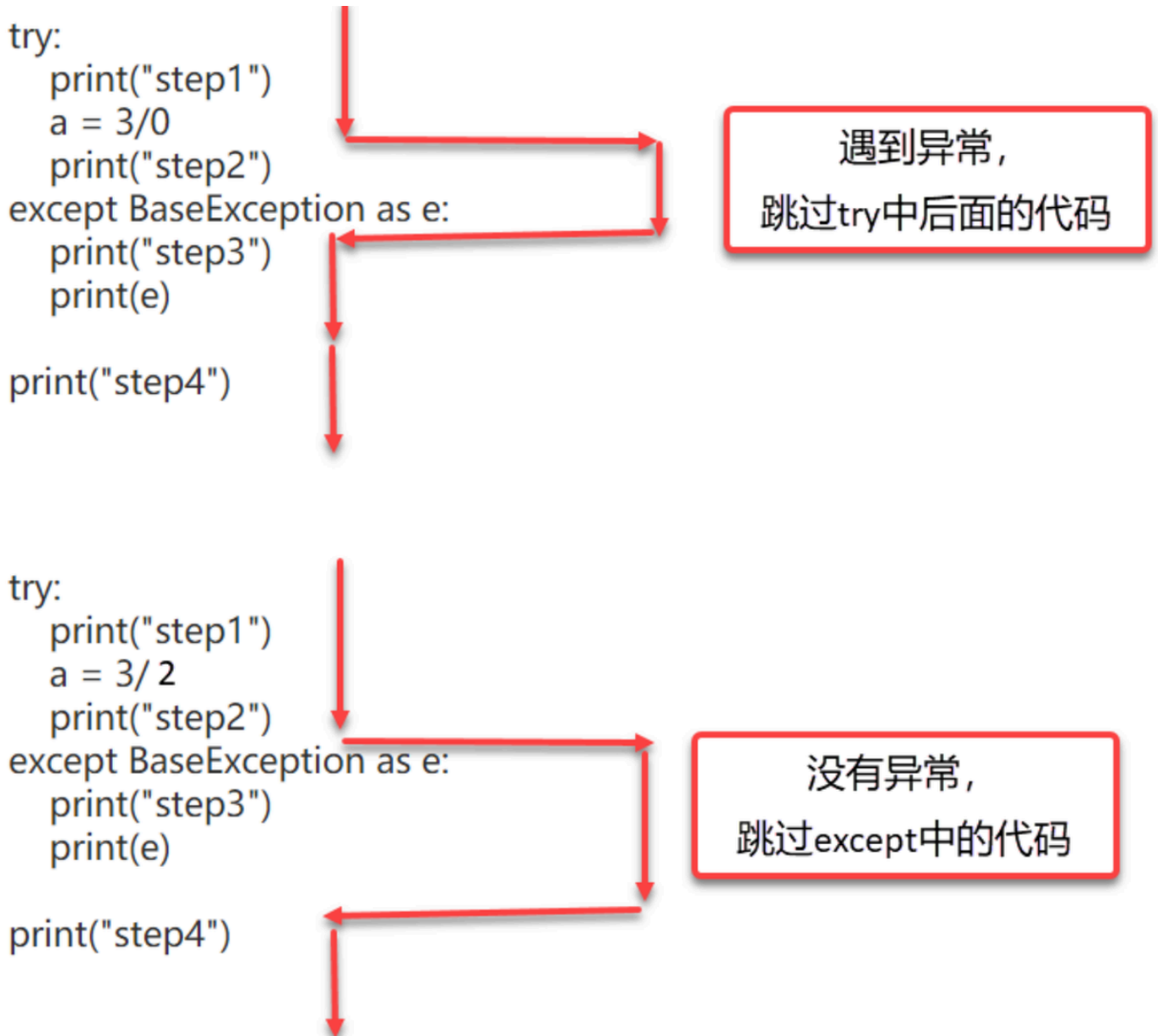
1.2 try 和 except 基本结构

`try...except` 是最常见的异常处理结构。结构语法如下：

代码块

```
1 try:
2     被监控的可能引发异常的语句块
3 except BaseException [as e]: # 把捕获到的具体异常信息赋值给变量 e
4     异常处理语句
```

1. `try` 块包含着可能引发异常的代码，`except` 块则用来捕捉和处理发生的异常；
2. 执行的时候，如果 `try` 块中没有引发异常，则跳过 `except` 块继续执行后续代码；
3. 执行的时候，如果 `try` 块中发生了异常，则跳过 `try` 块中的后续代码，跳到相应的 `except` 块中处理异常；异常处理后，继续执行后续代码。



1.3 try_多个 except 结构

一般建议尽量捕获可能出现的多个异常（按照**先子类后父类的顺序**），并且针对性的写出异常处理代码。为了避免遗漏可能出现的异常，可以在最后增加 `BaseException`。结构如下：

代码块

```
1  try:
2      被监控的、可能引发异常的语句块
3  except Exception1:
4      处理Exception1的语句块
5  except Exception2:
6      处理Exception2的语句块
7  [...]
8  except BaseException [as e]:
9      处理可能遗漏的异常的语句块
```

代码块

```
1  try:
2      a = input("请输入被除数: ")
3      b = input("请输入除数: ")
4      c = float(a)/float(b)
5      print(c)
6  except ZeroDivisionError:
7      print("异常: 除数不能为0")
8  except TypeError:
9      print("异常: 除数和被除数都应该为数值类型")
10 except BaseException as e:
11     print(e)
12     print(type(e))
```

1.4 try...except...else 结构

`try...except...else` 结构增加了 `else` 块。

如果 `try` 块中没有抛出异常，则执行 `else` 块。

如果 `try` 块中抛出异常，则执行 `except` 块，不执行 `else` 块。

总之，`else` 中的代码在没有抛出异常的情况下执行。

代码块

```
1  try:
2      a = input("请输入被除数: ")
3      b = input("请输入除数: ")
4      c = float(a)/float(b)
5  except BaseException as e:
6      print(e)
7  else:
8      print("除的结果是: ",c)
```

1.5 try...except...finally 结构

`try...except...finally` 结构中，`finally` 块无论是否发生异常都会被执行；通常用来释放 `try` 块中申请的资源。

代码块

```
1  try:
2      f = open("d:/a.txt", 'r')
3      content = f.readline()
4      print(content)
5  except BaseException as e:
6      print(e)
7  finally:
8      f.close()          # 释放资源。此处也可能会发生异常。若发生异常，则程序终止，不会继续往
                          # 下执行
9
10 print("step4")
11
```

- return 语句和异常处理问题

由于 return 有两种作用：结束方法运行、返回值。

一般不把 return 放到异常处理结构中，而是放到方法最后。

1.6 常见异常汇总

Python中的异常都派生自 `BaseException` 类。以下是一些常见的异常。

1. `SyntaxError`：语法错误

```
1      int a =3
2          ^
3  SyntaxError: invalid syntax
```

2. `NameError`：尝试访问一个没有声明的变量

```
1      print(a)
2  NameError: name 'a' is not defined
```


3. `ZeroDivisionError`: 除数为 0 错误

```
1      a = 3/0
2  ZeroDivisionError: division by zero
```

4. `ValueError`: 数值错误

```
1      float("gaoqi")
2  ValueError: could not convert string to
    float: 'gaoqi'
```

5. `TypeError`: 类型错误

```
1      123+"abc"
2  TypeError: unsupported operand type(s) for
    +: 'int' and 'str'
```

6. `AttributeError`: 访问对象的不存在的属性

```
1  a=100
2  a.sayhi()
3
4  AttributeError: 'int' object has no
    attribute 'sayhi'
```

7. `IndexError`: 索引越界异常

```
1 a = [4,5,6]
2 a[10]
3
4 IndexError: list index out of range
```

8、`KeyError`: 字典的关键字不存在

```
1 a = {'name': "gaoqi", 'age': 18}
2 a['salary']
3
4 KeyError: 'salary'
```

- 常见异常汇总

异常名称	说明
ArithmeticError	所有数值计算错误的基类
AssertionError	断言语句失败
AttributeError	对象没有这个属性
BaseException	所有异常的基类
DeprecationWarning	关于被弃用的特征的警告
EnvironmentError	操作系统错误的基类
EOFError	没有内建输入,到达EOF 标记
Exception	常规错误的基类
FloatingPointError	浮点计算错误
FutureWarning	关于构造将来语义会有改变的警告
GeneratorExit	生成器(generator)发生异常来通知退出
ImportError	导入模块/对象失败

IndentationError	缩进错误
IndexError	序列中没有此索引(index)
IOError	输入/输出操作失败
KeyboardInterrupt	用户中断执行(通常是输入^C)
KeyError	映射中没有这个键
LookupError	无效数据查询的基类
MemoryError	内存溢出错误(对于Python 解释器不是致命的)
NameError	未声明/初始化对象 (没有属性)
NotImplementedError	尚未实现的方法
OSError	操作系统错误
OverflowError	数值运算超出最大限制
OverflowWarning	旧的关于自动提升为长整型(long)的警告
PendingDeprecationWarning	关于特性将会被废弃的警告
ReferenceError	弱引用(Weak reference)试图访问已经垃圾回收了的对象
RuntimeError	一般的运行时错误
RuntimeWarning	可疑的运行时行为(runtime behavior)的警告
StandardError	所有的内建标准异常的基类
StopIteration	迭代器没有更多的值
SyntaxError	Python 语法错误
SyntaxWarning	可疑的语法的警告
SystemError	一般的解释器系统错误
SystemExit	解释器请求退出
TabError	Tab 和空格混用
TypeError	对类型无效的操作
UnboundLocalError	访问未初始化的本地变量

UnicodeDecodeError	Unicode 解码时的错误
UnicodeEncodeError	Unicode 编码时错误
UnicodeError	Unicode 相关的错误
UnicodeTranslateError	Unicode 转换时错误
UserWarning	用户代码生成的警告
ValueError	传入无效的参数
Warning	警告的基类
WindowsError	系统调用失败
ZeroDivisionError	除(或取模)零 (所有数据类型)

1.7 with 上下文管理

`finally` 块由于是否发生异常都会执行，通常我们放释放资源的代码。其实，可以通过 `with` 上下文管理，更方便的实现释放资源的操作。

`with` 上下文管理的语法结构如下：

代码块

```
1 with context_expr [ as var]:
2     语句块
```

`with` 上下文管理可以自动管理资源，在 `with` 代码块执行完毕后自动还原进入该代码之前的现场或上下文。不论何种原因跳出 `with` 块，不论是否有异常，总能保证资源正常释放。极大的简化了工作，在文件操作、网络通信相关的场合非常常用。

代码块

```
1 with open('./a.txt', 'r') as f:
2     for line in f:
3         print(line) # 输出文件中的每一行内容
```

1.8 Traceback 模块和生成异常日志

代码块

```
1 import traceback
2
```

```

3  try:
4      print("step1")
5      a = 1 / 0
6  except BaseException as e:
7      print(e)
8      with open("error.log", "a") as f:
9          traceback.print_exc(file=f) # 将异常信息写入 error.log 文件

```

1.9 自定义异常类

程序开发中，有时候需要自定义异常类。

自定义异常类一般都是运行时异常，通常**继承 Exception 或子类**即可。

命名一般以 **Error 和 Exception 为后缀**。

自定义异常由 **raise 语句主动抛出**。

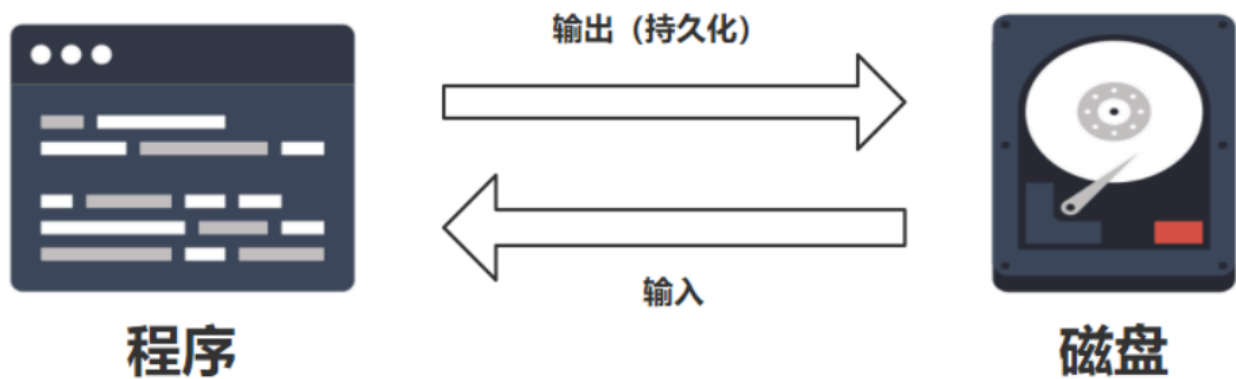
代码块

```

1  class AgeError(Exception): # 继承 Exception
2      def __init__(self, errorInfo):
3          '''
4              super().__init__(errorInfo)
5              1. 让父类保存异常信息到 e.args
6              2. 让默认的 str(e) 能输出异常信息
7          '''
8          super().__init__(errorInfo) # 调用父类的构造方法
9          self.errorInfo = errorInfo # 保存错误信息
10
11     def __str__(self):
12         return f'AgeError: {self.errorInfo}, age is invalid' # 返回错误信息
13
14  if __name__ == '__main__': #如果为True, 则模 块是作为独立文件运行, 可以执行测试代码
15     age = int(input("请输入年龄: "))
16     try:
17         if age < 0 or age > 120:
18             raise AgeError("年龄必须在0到120之间") # 抛出自定义异常
19         else:
20             print(f"年龄输入正确: {age}")
21     except AgeError as e:
22         print(e) # 捕获自定义异常并输出错误信息

```

2. 文件操作



2.1 file文件操作_操作系统底层关系_写入文件

1. 文本文件和二进制文件

按文件中数据组织形式，我们把文件分为**文本文件**和**二进制文件**两大类。

1. 文本文件

文本文件存储的是普通“字符”文本，python 默认为 unicode 字符集（两个字节表示一个字符，最多可以表示：65536个），可以使用记事本等文本程序打开。

2. 二进制文件

二进制文件把数据内容用“字节”进行存储，无法用记事本打开。必须使用专用的软件解码。常见的有：MP4视频文件、MP3音频文件、JPG图片、doc文档等等。

2. 文件操作相关模块概述

名称	说明
io模块	文件流的输入和输出操作 input output
os模块	基本操作系统功能，包括文件操作
glob模块	查找符合特定规则的文件路径名
fnmatch模块	使用模式来匹配文件路径名
fileinput模块	处理多个输入文件
filecmp模块	用于文件的比较
csv模块	用于csv文件处理
pickle和cPickle	用于序列化和反序列化
xml包	用于XML数据处理
bz2、gzip、zipfile、zlib、tarfile	用于处理压缩和解压缩文件（分别对应不同的算法）

3. 创建文件对象 open()

open() 函数用于创建文件对象，基本语法格式如下：

open(文件名 [, 打开方式]) 如果只是文件名，代表在当前目录下的文件。

文件名可以是相对路径和绝对路径。为了减少路径分隔符（windows 中是 \\），可以使用原始字符串：
r"D:\b.txt"。

打开方式有如下集中：

模式	描述
r	读 read模式
w	写 write模式。如果文件不存在则创建；如果文件存在，则重写新内容；
a	追加append模式。如果文件不存在则创建；如果文件存在，则在文件末尾追加内容
b	二进制binary模式（可与其他模式组合使用）
+	读、写模式（可与其他模式组合使用）

文本文件对象和二进制文件对象的创建：

1. 如果没有增加模式 b，则默认创建的是文本文件对象，处理的基本单元是“字符”。
2. 如果是二进制模式 b，则创建的是二进制文件对象，处理的基本单元是“字节”。

4. 文本文件的写入

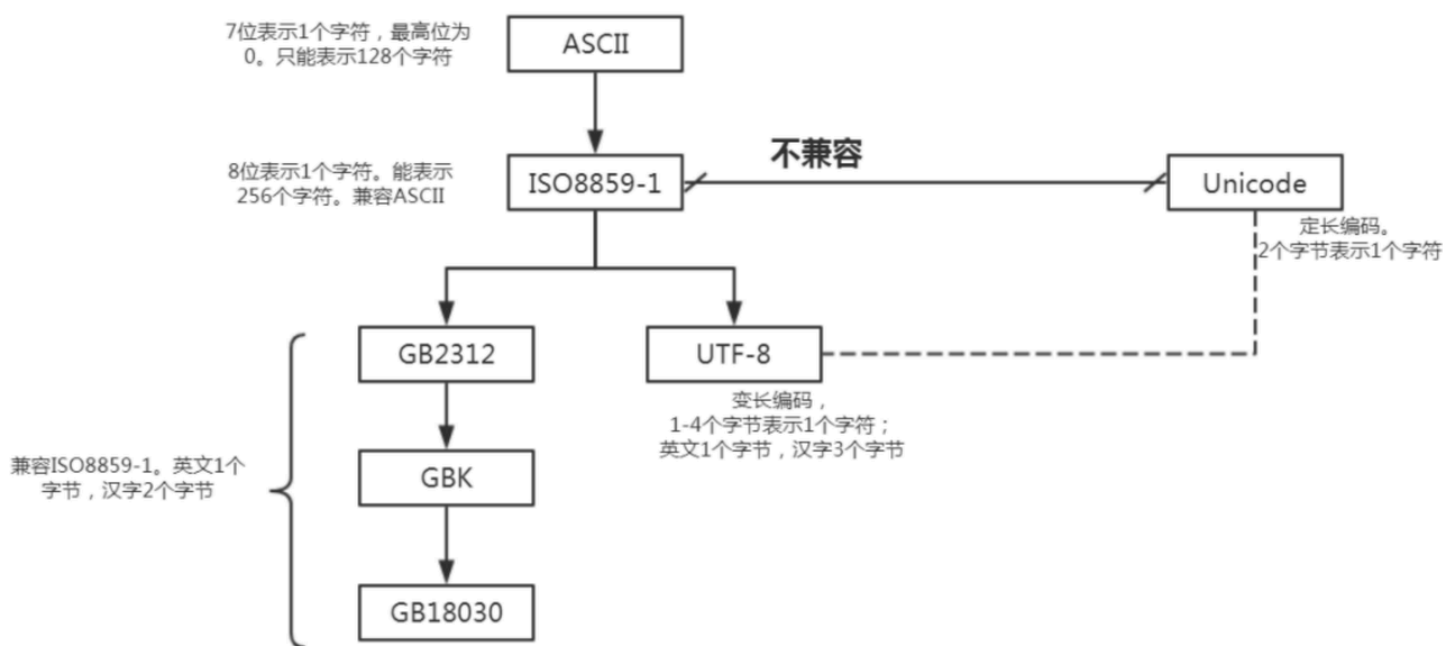
文本文件的写入一般就是三个步骤：

创建文件对象 -> 写入数据 -> 关闭文件对象

代码块

```
1 with open(r'a.txt', 'w') as f:
2     s = 'hello world'
3     f.write(s)
```

2.2 中文乱码问题



1. ASCII

全称为 **American Standard Code for Information Interchange**，美国信息交换标准代码，这是世界上最早最通用的单字节编码系统，主要用来显示现代英语及其他西欧语言。

2. GBK

全称为 **Chinese Internal Code Specification**，即汉字内码扩展规范，于1995年制定。它主要是扩展了 GB2312，在它的基础上又加了更多的汉字，它一共收录了21003个汉字。

3. Unicode

Unicode 编码设计成了 **固定两个字节**，所有的字符都用16位 ($2^{16}=65536$)表示，包括之前只占8位的英文字符等，所以会造成空间的浪费，UNICODE 在很长的一段时间内都没有得到推广应用。

Unicode 完全重新设计，不兼容 iso8859-1，也不兼容任何其他编码。

4. UTF-8

对于英文字母，unicode 也需要两个字节来表示。所以 unicode 不便于传输和存储。因此而产生了 UTF编码，UTF-8 全称是（8-bit Unicode Transformation Format）。

一般项目都会使用 UTF-8。

- 中文乱码问题

windows 操作系统默认的编码是 GBK，Linux 操作系统默认的编码是 UTF8。当用 open() 时，调用的是操作系统打开的文件，默认的编码是 GBK

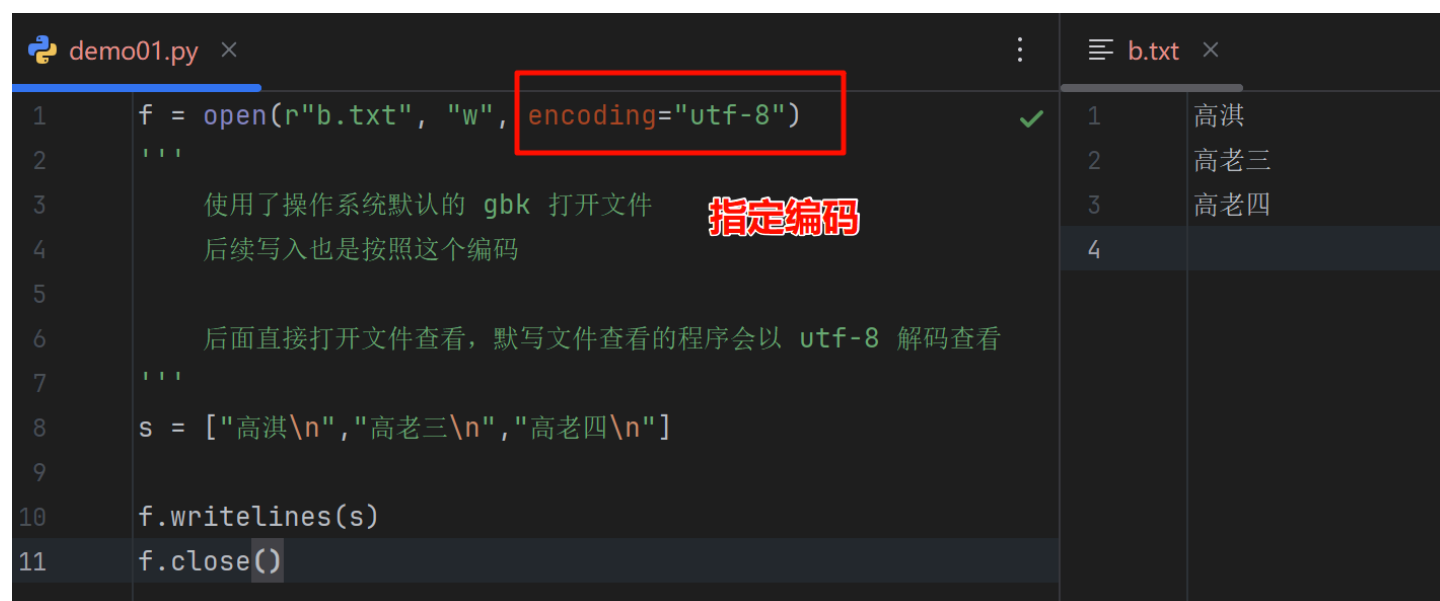


```
demo01.py x
1 f = open(r"b.txt", "w")
2 '''
3     使用了操作系统默认的 gbk 打开文件
4     后续写入也是按照这个编码
5
6     后面直接打开文件查看，默认文件查看的程序会以 utf-8 解码查看
7
8     s = ["高淇\n", "高老三\n", "高老四\n"]
9
10    f.writelines(s)
11    f.close()
```

b.txt x

! 该文件以错误的编码加载: 'UTF-8'

```
1
2
3
4
```



```
demo01.py x
1 f = open(r"b.txt", "w", encoding="utf-8")
2 '''
3     使用了操作系统默认的 gbk 打开文件
4     后续写入也是按照这个编码
5
6     后面直接打开文件查看，默认文件查看的程序会以 utf-8 解码查看
7
8     s = ["高淇\n", "高老三\n", "高老四\n"]
9
10    f.writelines(s)
11    f.close()
```

b.txt x

```
1 高淇
2 高老三
3 高老四
4
```

- write() / writelines() 写入数据

write(s) : 把字符串 s 写入到文件中

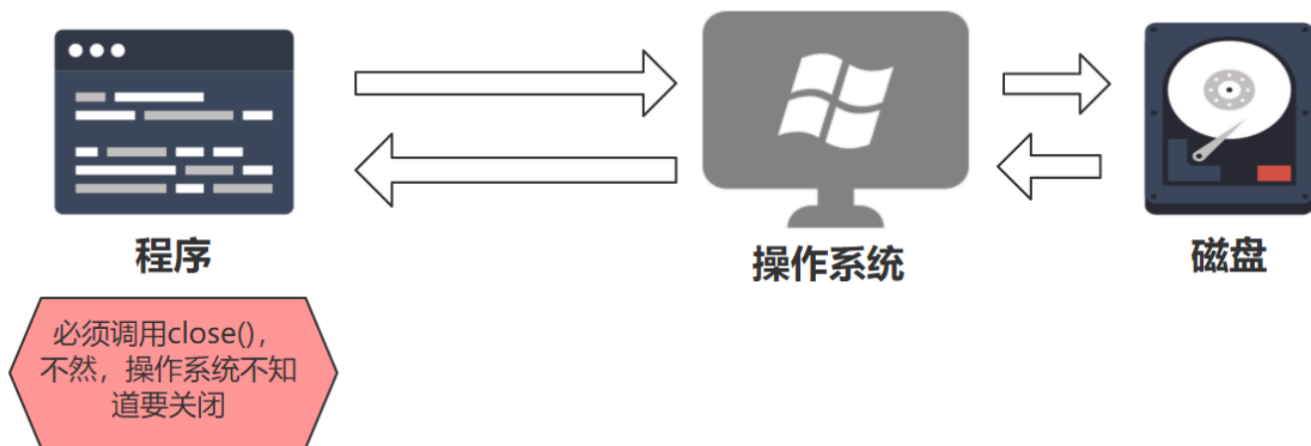
writelines(s) : 把字符串列表写入文件中，不会主动添加换行符

代码块

```
1 f = open(r"d:\bb.txt", "w", encoding="utf-8")
2 s = ["高淇\n", "高老三\n", "高老四\n"]
3
```

```
4 f.writelines(s)
5 f.close()
```

2.3 文件关闭流



由于文件底层是由操作系统控制，所以打开的文件对象必须显式调用 `close()` 方法关闭文件对象。当调用 `close()` 方法时，首先会把缓冲区数据写入文件(也可以直接调用 `flush()` 方法)，再关闭文件，释放文件对象。

为了确保打开的文件对象正常关闭，一般结合异常机制的 `finally` 或者 `with` 关键字实现无论何种情况都能关闭打开的文件对象。

`with`关键字（上下文管理器）可以自动管理上下文资源，不论什么原因跳出 `with`块，都能确保文件正确的关闭，并且可以在代码块执行完毕后自动还原进入该代码块时的现场。

代码块

```
1 # 1、使用 try...except...finally
2 try:
3     f = open("test.txt", "w")
4     f.write("hello world")
5 except BaseException as e:
6     print(e)
7 finally:
8     f.close()
9
10 # 2、使用 with 上下文管理器
11 with open("test.txt", "w") as f:
12     f.write("hello world")
```

2.4 文本文件的读取

文件的读取一般使用如下三个方法：

1. read([size])

从文件中读取 size 个字符，并作为结果返回。

如果没有 size 参数，则读取整个文件。读取到文件末尾，会返回空字符串。

2. readline()

读取一行内容作为结果返回。读取到文件末尾，会返回空字符串。

3. readlines()

文本文件中，每一行作为一个字符串存入列表，返回该列表。

1. 操作1：读取一个文件前 n 个字符



```
demo01.py x
1 with open('b.txt', 'r') as f:
2     s = f.read(4)
3     print(s)

运行 demo01 x
D:\python\py_project\PythonProject\pro3\.venv\Scripts\python.exe D:\python\py
abcd
```

2. 操作2：文件较小，一次将文件内容读入程序中



```
demo01.py x
1 with open('b.txt', 'r') as f:
2     s = f.read()
3     print(s)

运行 demo01 x
D:\python\py_project\PythonProject\pro3\.venv\Scripts\python.exe D:\python\p
abcdef
123456
```

3. 操作3：按行读取一个文件

The screenshot shows a code editor with two tabs: `demo01.py` and `b.txt`. The `demo01.py` tab is active and contains the following Python code:

```
1 with open('b.txt', 'r') as f:
2     while True:
3         line = f.readline()
4         if not line:
5             break
6
7     print(line, end='')
```

A green checkmark is visible next to line 1. The `b.txt` tab shows the file's content:

```
1 abcdef
2 123456
3
```

Below the code editor, there is a terminal window titled "运行 demo01". It shows the command `D:\python\py_project\PythonProject\pro3\.venv\Scripts\python.exe D:\python\` and the output:

```
abcdef
123456
```

4. 使用迭代器（每次返回一行）读取文本文件

The screenshot shows the same code editor with the `demo01.py` tab active. The code is now using a `for` loop:

```
1 with open('b.txt', 'r') as f:
2     for line in f:
3         print(line, end='')
```

A lightbulb icon is visible next to line 3. The `b.txt` tab shows the same content as before:

```
1 abcdef
2 123456
3
```

The terminal window below shows the same command and output as in the previous screenshot:

```
D:\python\py_project\PythonProject\pro3\.venv\Scripts\python.exe D:\python\
abcdef
123456
```

5. 为文本文件的每一行的行首增加行号

代码块

```
1 with open("b.txt", 'r', encoding='utf-8') as f:
2     lines = f.readlines()
3
4     lines2 = ['{0}\t{1}'.format(index, line) for index, line in zip(range(1,
5 len(lines) + 1), lines)]
6
```

```
7 with open("bx.txt", 'w', encoding='utf-8') as f:
8     f.writelines(lines2)
```

2.5 二进制文件读写_文件拷贝

二进制文件的处理流程和文本文件流程一致。首先还是要创建文件对象，不过，我们需要指定二进制模式，从而创建出二进制文件对象。例如：

```
f = open(r"d:\a.txt", 'wb') # 可写的、重写模式的二进制文件对象
f = open(r"d:\a.txt", 'ab') # 可写的、追加模式的二进制文件对象
f = open(r"d:\a.txt", 'rb') # 可读的二进制文件对象
```

创建好二进制文件对象后，仍然可以使用 `write()`、`read()` 实现文件的读写操作。

代码块

```
1 with open('log.png', 'r') as srcFile, open('log2.txt', 'w') as dstFile:
2     for line in srcFile:
3         dstFile.write(line)
```

2.6 文件对象的常用属性和方法_文件任意位置操作

文件对象封装了文件相关的操作。

文件对象的属性

属性	说明
name	返回文件的名字
mode	返回文件的打开模式
closed	若文件被关闭, 则返回True

代码块

```
1 with open('b.txt', 'w') as f:
2     print('file.name =', f.name) # file.name = b.txt
3     print('file.mode =', f.mode) # file.mode = w
```

```
4 print('file.closed =', f.closed) # file.closed = False
```

文件对象的打开模式

模式	说明
r	读模式
w	写模式
a	追加模式
b	二进制模式（可与其他模式组合）
+	读写模式（可以其他模式组合）

文件对象的常用方法

方法名	说明
read([size])	从文件中读取size个字节或字符的内容返回。若省略[size]，则读取到文件末尾，即一次读取文件所有内容
readline()	从文本文件中读取一行内容
readlines()	把文本文件中每一行都作为独立的字符串对象，并将这些对象放入列表返回
write(str)	将字符串str内容写入文件
writelines(s)	将字符串列表s写入文件文件，不添加换行符
seek(offset[,whence])	把文件指针移动到新的位置，offset表示相对于whence的多少个字节的偏移量；offset: off为正往结束方向移动，为负往开始方向移动whence不同的值代表不同含义：0: 从文件头开始计算（默认值）1: 从当前位置开始计算2: 从文件尾开始计算
tell()	返回文件指针的当前位置
truncate([size])	不论指针在什么位置，只留下指针前size个字节的内容，其余全部删除；如果没有传入size，则当指针当前位置到文件末尾内容全部删除
flush()	把缓冲区的内容写入文件，但不关闭文件
close()	把缓冲区内容写入文件，同时关闭文件，释放文件对象相关资源

代码块

```
1  # 文件内容
2  with open('b.txt', 'r', encoding='utf-8') as f:
3      print(f.tell()) # 0
4      print("读取的内容是 = {}".format(str(f.readline()))) # 读取的内容是 = abcdef
5      print(f.tell()) # 6
6      f.seek(3, 0) # 相对文件开头偏移3个位置
7      print("读取的内容是 = {}".format(str(f.readline()))) # 读取的内容是 = def
```

2.7 使用 pickle 序列化和反序列化

序列化：将对象转化成“串行化”数据形式，存储到硬盘或通过网络传输到其他地方。

反序列化：是指相反的过程，将读取到的“串行化数据”转化成对象。

可以使用 pickle 模块中的函数，实现序列化和反序列操作。

Python中，一切皆对象，对象本质上就是一个“存储数据的内存块”。有时候，需要将“内存块的数据”保存到硬盘上，或者通过网络传输到其他的计算机上。这时候，就需要“对象的序列化和反序列化”。

对象的序列化机制广泛的应用在分布式、并行系统上。

序列化和反序列化：

- `pickle.dump(obj, file)` `obj` 就是要被序列化的对象，`file` 指的是存储的文件
- `pickle.load(file)` 从 `file` 读取数据，反序列化成对象

代码块

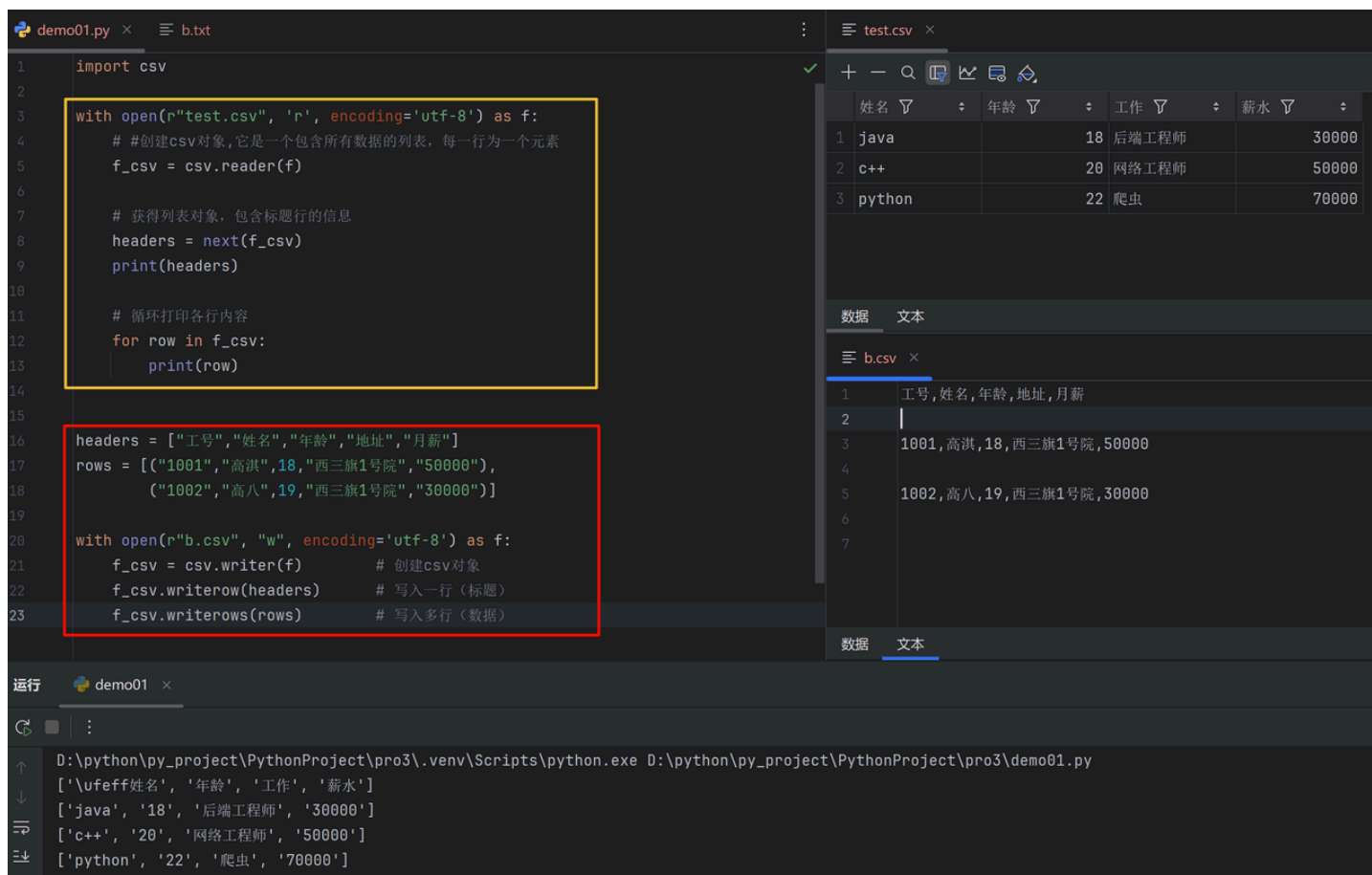
```
1  import pickle
2
3  with open('data.dat', 'wb') as f:
4      data = {
5          "name": "python语言",
6          "age": 22,
7          "scores": {
8              "语文": 97,
9              "数学": 98,
10             "英语": 99
11         }
12     }
13     pickle.dump(data, f)
14
15 with open('data.dat', 'rb') as f:
16     data = pickle.load(f)
17     print(data)
```

2.8 CSV 文件操作

csv是逗号分隔符文本格式，常用于数据交换、Excel文件和数据库数据的导入和导出。

与Excel文件不同，CSV文件中：

- 值没有类型，**所有值都是字符串**
- 不能指定字体颜色等样式
- 不能指定单元格的宽高，不能合并单元格
- 没有多个工作表
- 不能嵌入图像图表



2.9 os 模块调用系统可执行命令

- `os.system` 可以帮助直接调用系统的命令。

【示例】 `os.system` 调用windows系统的记事本程序

```
1 import os
2 os.system("notepad.exe")
```

【示例】 `os.system` 调用windows系统中ping命令

```
1 import os
2 os.system("ping www.baidu.com")
```

- `os.startfile` 可以直接调用可执行程序

代码块

```
1 import os
2
3 os.startfile(r"C:\Program Files (x86)\Tencent\WeChat\WeChat.exe")
```

2.10 os 模块文件和目录操作

可以通过前面讲的文件对象实现对于文件内容的读写操作。如果，还需要对文件和目录做其他操作，可以使用 `os` 和 `os.path` 模块。

`os` 模块操作文件、目录本身，而不是对文件内容操作。

os 模块下常用操作文件的方法

方法名	描述
<code>remove(path)</code>	删除指定的文件
<code>rename(src,dest)</code>	重命名文件或目录
<code>stat(path)</code>	返回文件的所有属性
<code>listdir(path)</code>	返回path目录下的文件和目录列表

方法名	描述
mkdir(path)	创建目录
makedirs(path1/path2/path3/...)	创建多级目录
rmdir(path)	删除目录
removedirs(path1/path2...)	删除多级目录
getcwd()	返回当前工作目录：current work dir
chdir(path)	把path设为当前工作目录
walk()	遍历目录树
sep	当前操作系统所使用的路径分隔符

代码块

```
1  import os
2
3  print(os.name) # windows -> nt    linux -> posix
4  print(os.sep) # windows -> \    linux -> /
5  print(repr(os.linesep)) # windows -> '\r\n'    linux -> \n
6
7  a = '3'
8  print(a)
9  print(repr(a)) # repr 可以显示数据信息
10
11 # 获取文件和文件夹的相关信息
12 print(os.stat("b.txt"))
13
14 print(os.getcwd()) # 获得当前工作目录
15 os.chdir("D:\\") # 当前的工作目录就变成了d: 的根目录
16 print(os.getcwd()) # 获得当前工作目录
17
18 dirs = os.listdir("D:\\")
19 print(dirs)
```

2.11 os.path 模块

os.path 模块判断文件和目录的路径。

`os.path` 模块提供了目录相关（路径判断、路径切分、路径连接、文件夹遍历）的操作

方法	描述
<code>isabs(path)</code>	判断path是否绝对路径
<code>isdir(path)</code>	判断path是否为目录
<code>isfile(path)</code>	判断path是否为文件
<code>exists(path)</code>	判断指定路径的文件是否存在
<code>getsize(filename)</code>	返回文件的大小
<code>abspath(path)</code>	返回绝对路径
<code>dirname(p)</code>	返回目录的路径
<code>getatime(filename)</code>	返回文件的最后访问时间
<code>getmtime(filename)</code>	返回文件的最后修改时间
<code>walk(top,func,arg)</code>	递归方式遍历目录
<code>join(path,*paths)</code>	连接多个path
<code>split(path)</code>	对路径进行分割，以列表形式返回
<code>splittext(path)</code>	从路径中分割文件的扩展名

代码块

```
1  import os
2  import os.path
3  ##### 获得目录、文件基本信息 #####
4  print(os.path.isabs("d:/a.txt"))    #是否绝对路径
5  print(os.path.isdir("d:/a.txt"))    #是否目录
6  print(os.path.isfile("d:/a.txt"))    #是否文件
7  print(os.path.exists("a.txt"))      #文件是否存在
8  print(os.path.getsize("a.txt"))      #文件大小
9  print(os.path.abspath("a.txt"))      #输出绝对路径
10 print(os.path.dirname("d:/a.txt"))   #输出所在目录
11
12 ##### 获得创建时间、访问时间、最后修改时间 #####
13 print(os.path.getctime("a.txt"))     #返回创建时间
14 print(os.path.getatime("a.txt"))     #返回最后访问时间
15 print(os.path.getmtime("a.txt"))     #返回最后修改时间
```

```

16
17 ##### 对路径进行分割、连接操作 #####
18 path = os.path.abspath("a.txt")      #返回绝对路径
19
20 print(os.path.splitext(path))        #返回元组：路径、扩展名
21 # ('C:\\Users\\Administrator\\PycharmProjects\\mypro_io\\test_os\\a', '.txt')
22
23 print(os.path.join("aa","bb","cc"))  #返回路径：aa/bb/cc

```

- 列出指定目录下所有的 .py 文件，并输出文件名

代码块

```

1  import os
2  import os.path
3
4  path = os.getcwd()
5  files = os.listdir(path)
6
7  # 1、通过字符串查找 py 判断
8  for file in files:
9      pos = file.rfind('.')
10     if file[pos + 1:] == 'py': # 获取扩展名, 如果是 py 就输出
11         print(file, end = '\t')
12
13 print()
14
15 # 2、直接用 endswith 函数判断
16 files = [file for file in files if file.endswith('.py')]
17 for file in files:
18     print(file, end = '\t')

```

2.12 walk() 递归遍历所有文件和目录

os.walk() 方法是一个简单易用的文件、目录遍历器，可以帮助我们高效的处理文件、目录方面的事情。格式如下：

```
os.walk(top[, topdown=True[, onerror=None[, followlinks=False]]])
```

其中，top: 是要遍历的目录。topdown: 可选，True，遍历 top 目录再遍历子目录。返回三元组 (root、dirs、files):

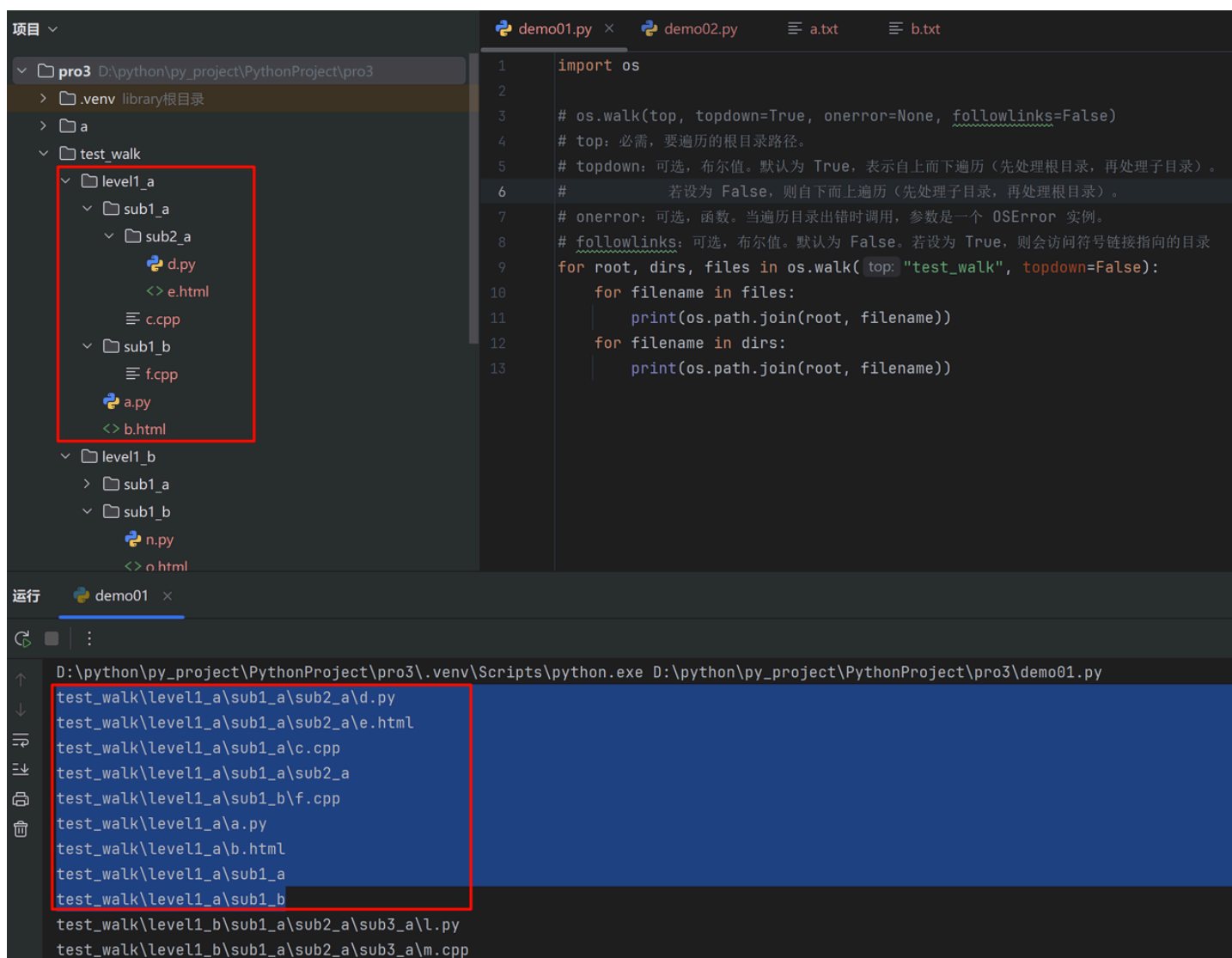
root：当前正在遍历的文件夹本身

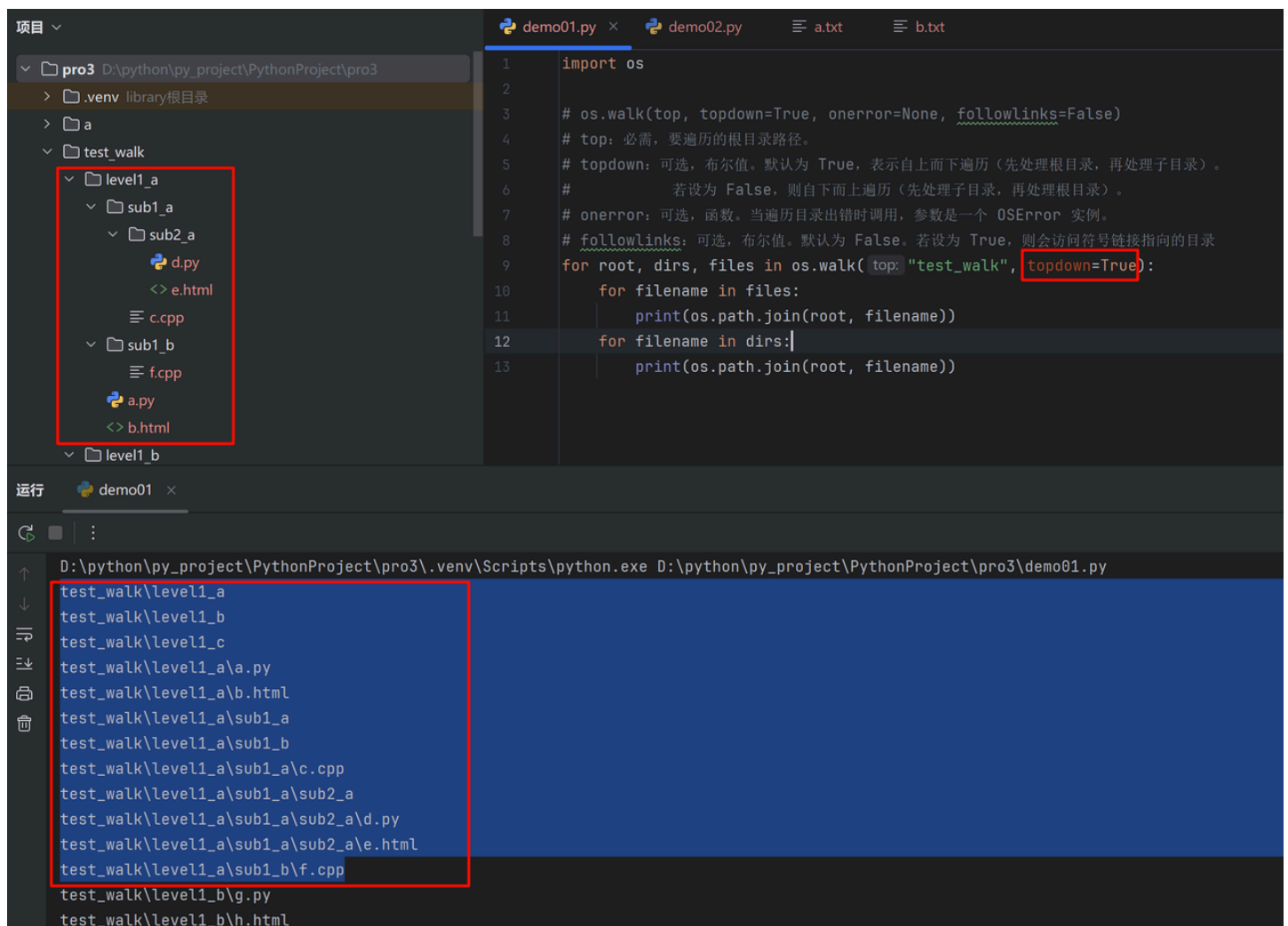
dirs：一个列表，该文件夹中所有的目录的名字

files：一个列表，该文件夹中所有的文件的名字

代码块

```
1 import os.path
2
3 path = os.getcwd()
4 content = os.walk(path, topdown=False)
5
6 for root, dirs, files in content:
7     for name in files:
8         print(os.path.join(root, name))
9     for name in dirs:
10        print(os.path.join(root, name))
```





2.13 shutil 模块（拷贝和压缩）

shutil 模块是python标准库中提供的，主要用来做文件和文件夹的拷贝、移动、删除等；还可以做文件和文件夹的压缩、解压缩操作。

os 模块提供了对目录或文件的一般操作。**shutil 模块作为补充**，提供了移动、复制、压缩、解压等操作，这些 os 模块都没有提供。

代码块

```
1 import shutil
2
3 # copy文件内容
4 shutil.copyfile("a.txt", "a_copy.txt")
```

代码块

```
1 import shutil
2
3 # "音乐"文件夹不存在才能用。
4 # 将文件夹“电影/学习”下面的内容拷贝到文件夹“音乐”下。
5 # 拷贝时忽略所有的 html 和 htm 文件。
```

```
6 shutil.copytree("电影/学习", "音乐", ignore=shutil.ignore_patterns("*.html",  
    "*.htm"))
```

【示例】 实现将文件夹所有内容压缩(使用 `shutil` 模块)

```
1 import shutil  
2 import zipfile  
3 #将"电影/学习"文件夹下所有内容压缩到"音乐2"文件夹下  
  生成movie.zip  
4 #shutil.make_archive("音乐2/movie","zip","电  
  影/学习")  
5  
6 #压缩:将指定的多个文件压缩到一个zip文件  
7 #z = zipfile.ZipFile("a.zip","w")  
8 #z.write("1.txt")  
9 #z.write("2.txt")  
10 #z.close()
```

【示例】 实现将压缩包解压缩到指定文件夹(使用 `shutil` 模块)

```
1 import shutil  
2 import zipfile  
3 #解压缩:  
4 z2 = zipfile.ZipFile("a.zip","r")  
5 z2.extractall("d:/") #设置解压的地址  
6 z2.close()
```

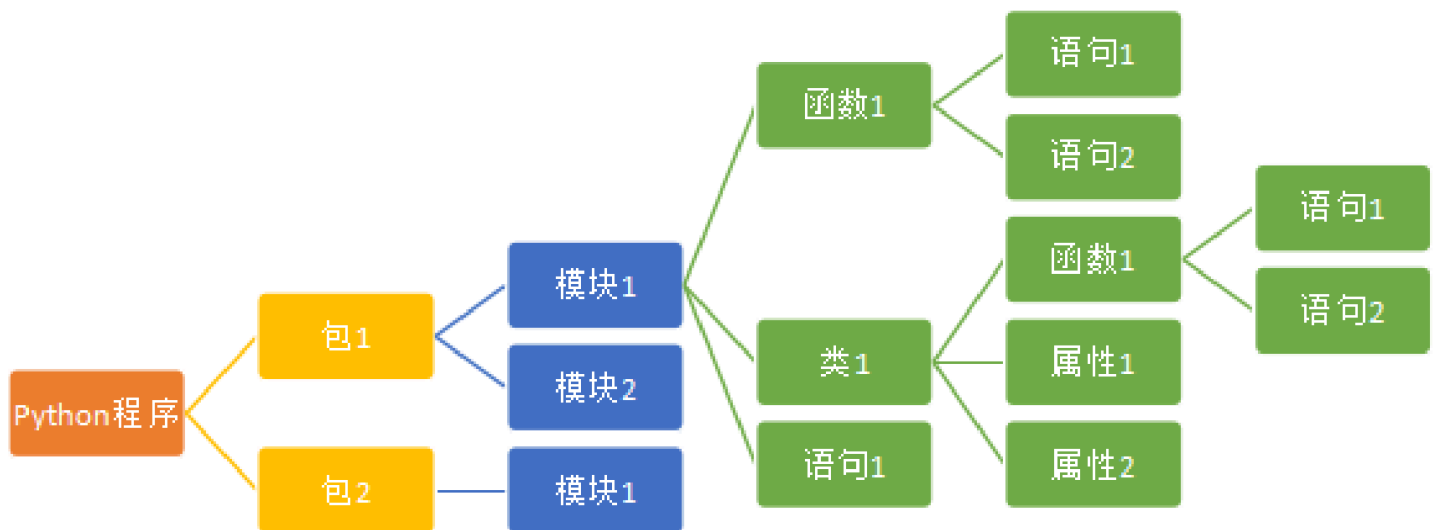
2.14 递归遍历目录下的所有文件和目录

代码块

```
1 import os
2 import os.path
3
4 # 递归遍历目录树
5 def my_print_file(path, level):
6     child_files = os.listdir(path)
7     for file in child_files:
8         file_path = os.path.join(path, file)
9         print("\t" * level + file_path[file_path.rfind(os.sep) + 1:])
10        if os.path.isdir(file_path):
11            my_print_file(file_path, level + 1)
12
13 my_print_file("D:\\", 0)
```

3. 模块

3.1 模块化编程理念_什么是模块_哲学思想



1. Python程序由模块组成。一个模块对应python源文件，一般后缀名是: `.py`
 2. 模块由语句组成。运行Python程序时，按照模块中语句的顺序依次执行
 3. 语句是Python程序的构造单元，用于创建对象、变量赋值、调用函数、控制语句等
- 标准库模块 (standard library)

与函数类似，**模块也分为标准库模块和用户自定义模块。**

Python标准库提供了操作系统功能、网络通信、文本处理、文件处理、数学运算等基本功能。比如: random(随机数)、math(数学 运算)、time(时间处理)、file(文件处理)、os(和操作系统交互)、

sys(和解释器交互)等。

另外，Python还提供了海量的第三方模块，使用方式和标准库类似。功能覆盖了我们能想象到的所有领域，比如：科学计算、WEB 开发、大数据、人工智能、图形系统等。

- 为什么需要模块化编程

模块(module)对应于Python源代码文件(.py文件)。模块中可以定义变量、函数、类、普通语句。这样，我们可以将一个Python程序分解成多个模块，便于后期的重复应用。

模块化编程（Modular Programming）将一个任务分解成多个模块。每个模块就像一个积木一样，便于后期的反复使用、反复搭建。模块化编程有以下优势：

1. 便于将一个任务分解成多个模块，实现团队协作开发，完成大规模程序
2. 实现代码复用。一个模块实现后，可以被反复调用
3. 可维护性增强

3.2 模块化编程的流程_设计和实现的分离思想

1. 模块化编程的一般流程

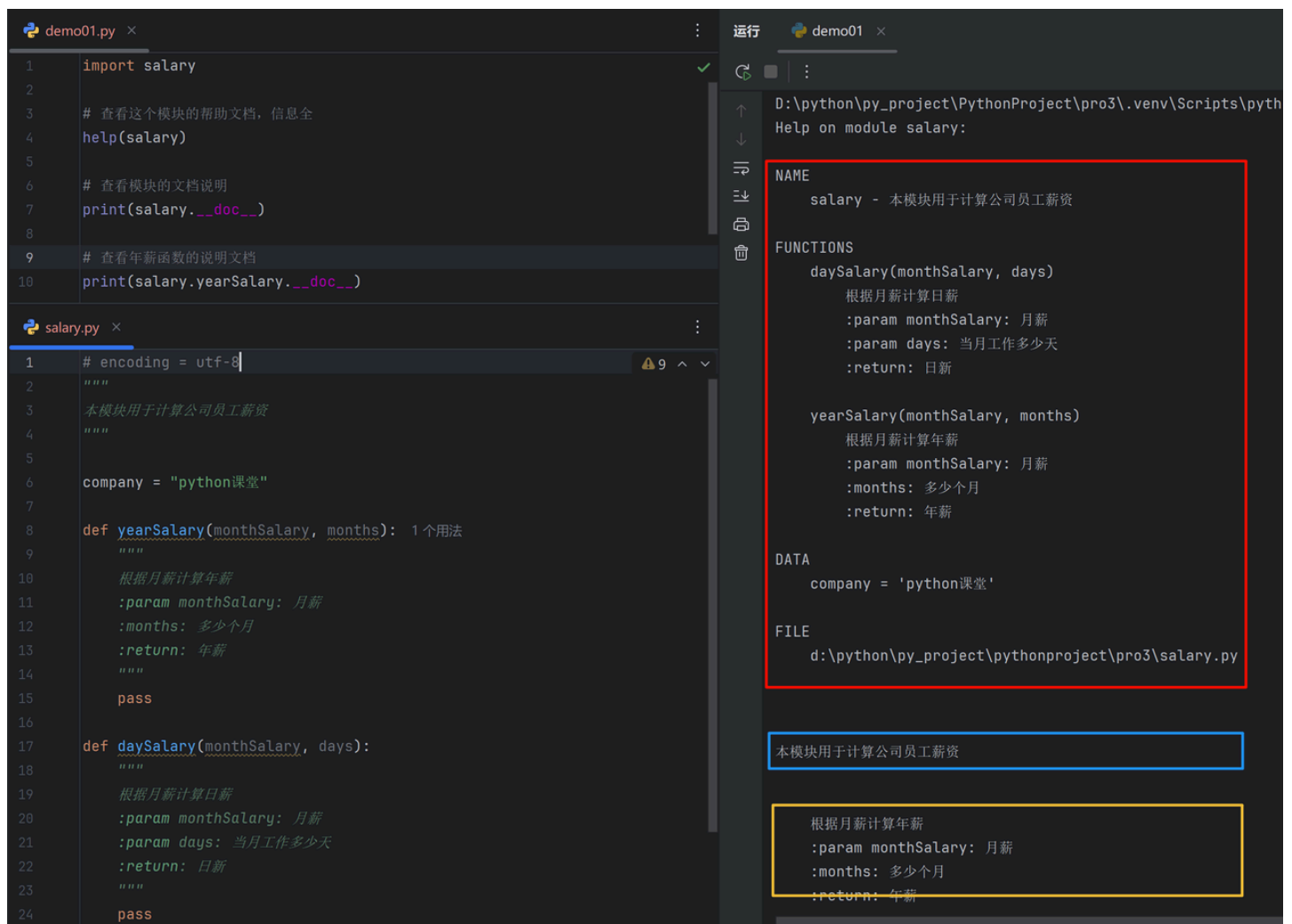
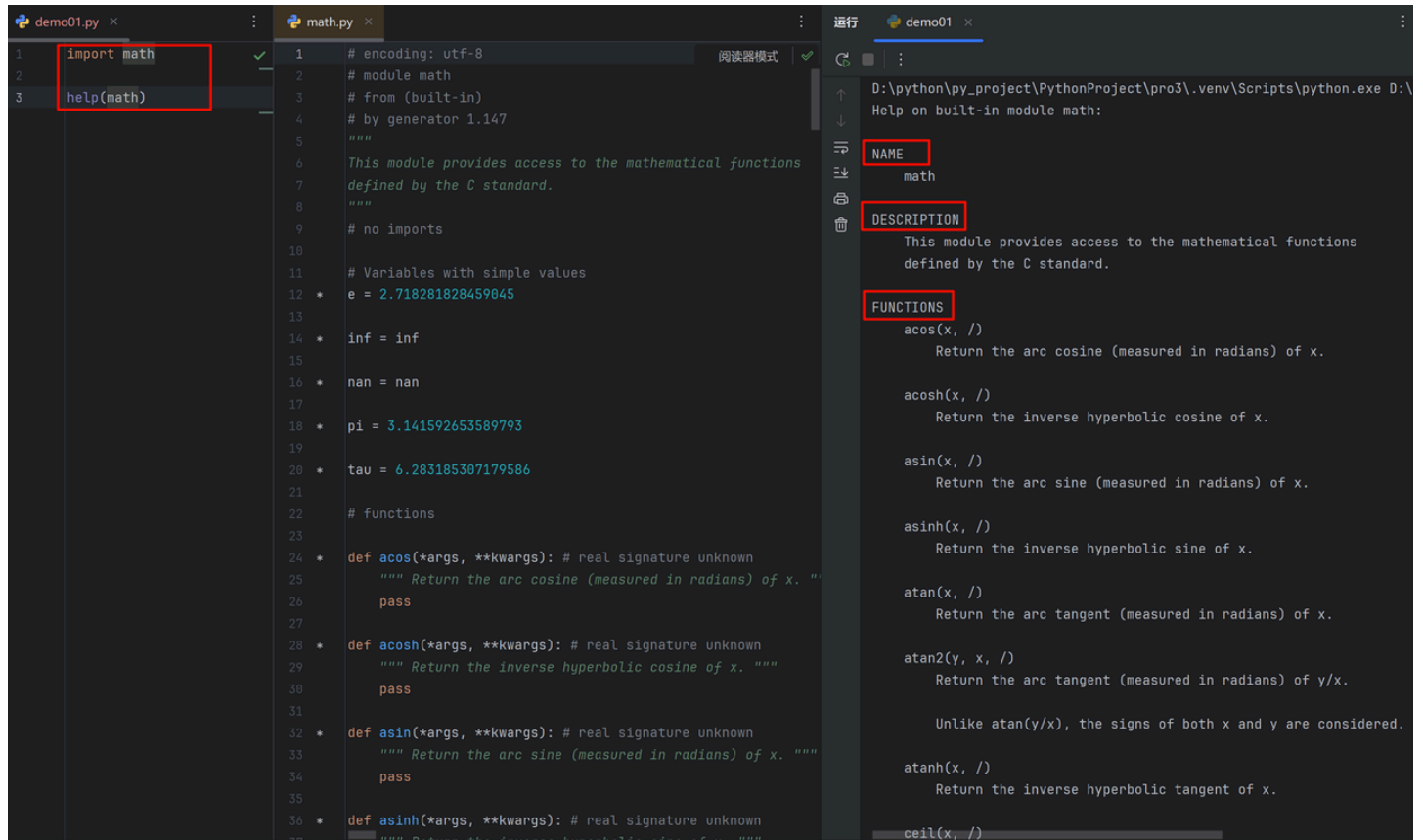
1. 设计API，进行功能描述
2. 编码实现API中描述的功能
3. 在模块中编写测试代码，并消除全局代码
4. 使用私有函数实现不被外部客户端调用的模块函数

2. 模块的API和功能描述要点

API(Application Programming Interface 应用程序编程接口)是用于描述模块中提供的函数和类的功能描述和使用方式描述。

模块化编程中，首先设计的就是模块的API（即要实现的功能描述），然后开始编码实现API中描述的功能。最后，在其他模块中导入本模块进行调用。

可以通过 `help(模块名)` 查看模块的API。一般使用时先导入模块 然后通过 `help` 函数查看。



- 通过 `__name__ == "__main__"` 独立处理模块的测试代码

```

1'''# encoding = utf-8
2    """
3    本模块用于计算公司员工薪资
4    """
5
6    company = "python课堂"
7
8    def yearSalary(monthSalary, months):
9        """
10        根据月薪计算年薪
11        :param monthSalary: 月薪
12        :months: 多少个月
13        :return: 年薪
14        """
15        return monthSalary * months
16
17    def daySalary(monthSalary, days):
18        """
19        根据月薪计算日薪
20        :param monthSalary: 月薪
21        :param days: 当月工作多少天
22        :return: 日薪
23        """
24        return monthSalary / days
25
26    '''
27    __name__: 是一个内置的特殊变量（也称为“魔术变量”），用于表示当前模块的名称。
28    而 __main__ 是 Python 顶层执行环境的名称。
29
30    __name__ 取值规则：
31    1、当 Python 文件被直接运行（比如 python salary.py）时，该模块的 __name__ 会
    被自动赋值为字符串 '__main__'。
32    2、当 Python 文件被作为模块导入（比如 import salary）时，该模块的 __name__ 会
    被赋值为模块名（即文件名去掉 .py 后缀，如 'salary'）。
33
34    if __name__ == '__main__': 的作用：
35    判断当前模块是否正在被直接执行，而不是被导入到其他模块中。
36    这样可以在模块做本模块的测试代码
37
38    如果条件成立（即直接运行），则执行该代码块内的语句。
39    如果条件不成立（即被导入），则跳过该代码块，避免在执行 import 时产生意外副作用
    （例如执行测试、启动程序等）。
40    '''
41    if __name__ == '__main__':
42        print(yearSalary(20000, 16)) # 如果是在 salary 模块运行，就会执行这里面的语句

```

在正常情况下，模块名字对应源文件名。仅有一个例外，就是当一个模块被作为程序入口时（主程序、交互式提示符下），它的 `__name__` 的值为 `__main__`。我们可以根据这个特点，将模块源代码文件中的测试代码进行独立的处理。

3.3 模块导入

模块化设计的好处之一就是“代码复用性高”。写好的模块可以被反复调用，重复使用。模块的导入就是“在本模块中使用其他模块”。

3.3.1 import 语句导入模块

- `import` 语句的基本格式如下：

代码块

```
1  import 模块名                #导入一个模块
2  import 模块1, 模块2 [,...]   #导入多个模块
3  import 模块名 as 模块别名    #导入模块并使用新名字
```

- `import` 加载的模块分为四种类型

1. 使用python编写的代码 .py 文件
2. 已被编译为共享库或 DLL 的C或C++扩展
3. 一组模块的包
4. 使用C编写并链接到python解释器的内置模块

一般通过 `import` 语句实现模块的导入和使用，`import` 本质上是使用了内置函数 `__import__()`。

通过 `import` 导入一个模块时，python解释器进行执行，最终会生成一个对象，这个对象就代表了被加载的模块。

代码块

```
1  import salary
2  import math
3
4  # 1、导入自定义模块
5  print(salary) # <module 'salary' from
                 'D:\\python\\py_project\\PythonProject\\pro3\\salary.py'>
6  print(id(salary)) # 1982533865776
7  print(type(salary)) # <class 'module'>
8
9  # 2、导入标准库模块
10 print(math) # <module 'math' (built-in)>
11 print(id(math)) # 1982533868496
```

```
12 print(type(math)) # <class 'module'>
```

由上述代码可以看到 `math` 模块被加载后，实际会生成一个 `module` 类的对象，该对象被 `math` 变量引用。可以通过 `math` 变量引用模块中所有的内容。

可以通过 `import` 导入多个模块，本质上也是生成多个 `module` 类的对象而已。

有时候，需要给模块起一个别名，本质上，这个别名仅仅是新创建一个变量引用加载的模块对象而已。使用 `as` 即可起一个别名。

代码块

```
1 import math as m
2
3 '''
4     import math as m
5     相当于
6     import math
7     m = math
8     math 不能再被使用，必须使用 m 别名
9 '''
10 print(m.e)
```

3.3.2 from...import 导入

python中可以使用 `from...import` 导入模块中的成员。基本语法：

```
from 模块名 import 成员1, 成员2, ...
```

如果需要导入一个模块中的所有成员，可以： `from 模块名 import *`

尽量避免 `from 模块名 import *` 这种写法。`*` 表示导入模块中所有的不是以下划线 `_` 开头的名字都导入到当前位置。但是不知道导入了什么名字，很有可能覆盖已经定义的名字。而且可读性极差，生产环境禁止使用。

代码块

```
1 from math import pi, sin
2
3 print(sin(pi / 2))
4
5 # print(math.e) # 不能使用其它成员，NameError: name 'math' is not defined
```

3.3.3 import语句和from...import语句的区别

`import` 导入的是模块。 `from...import` 导入的是模块中的函数/类。

```
代码块
import salary
2
3 print(salary.yearSalary(20000, 16)) # 需要使用模块名引用其中的内容
```

代码块

```
1 from salary import *
2
3 print(yearSalary(20000, 16)) # 直接使用 salary 模块中的函数
```

3.3.4 __import__() 动态导入

`import` 语句本质上就是调用内置函数 `__import__()`，可以通过它实现动态导入。给 `__import__()` 动态传递不同的参数值，就能导入不同的模块。

代码块

```
1 s = "math"
2
3 m = __import__(s) # 导入后生成的模块对象的引用给变量 m
4
5 print(m.sin(m.pi / 2))
```

一般不建议我们自行使用 `__import__()` 导入，其行为在python2 和python3中有差异，会导致意外错误。如果需要动态导入可以使用 `importlib` 模块。

代码块

```
1 import importlib
2
3 # 动态导入 salary 模块
4 m = importlib.import_module('salary')
5
6 print(m.yearSalary(20000, 16))
```

3.3.5 模块的加载问题

当导入一个模块时，模块中的代码都会被执行。不过，如果再次导入这个模块，则不会再次执行。

一个模块无论导入多少次，这个模块在整个解释器进程内有且仅有一个实例对象。

有的时候确实需要重新加载一个模块，这时候可以使用：`importlib.reload()` 方法：

```
demo01.py x
1 import salary
2 import salary
3 import importlib
4
5 print("test")
6
7 importlib.reload(salary)
8
9 print("test2")
10

salary.py x
1 # encoding = utf-8
2 """
3 本模块用于计算公司员工薪资
4 """
5
6 print("salary module load")
7
8 company = "python课堂"
9
10 def yearSalary(monthSalary, months):
11     """
12     根据月薪计算年薪
13     :param monthSalary: 月薪
14     :param months: 多少个月
```

运行 demo01 x

```
D:\python\py_project\PythonProject\pro3\.venv\Scripts\python.exe D:\python\py_project
salary module load
test
salary module load
test2
```

3.4 包 package 的使用

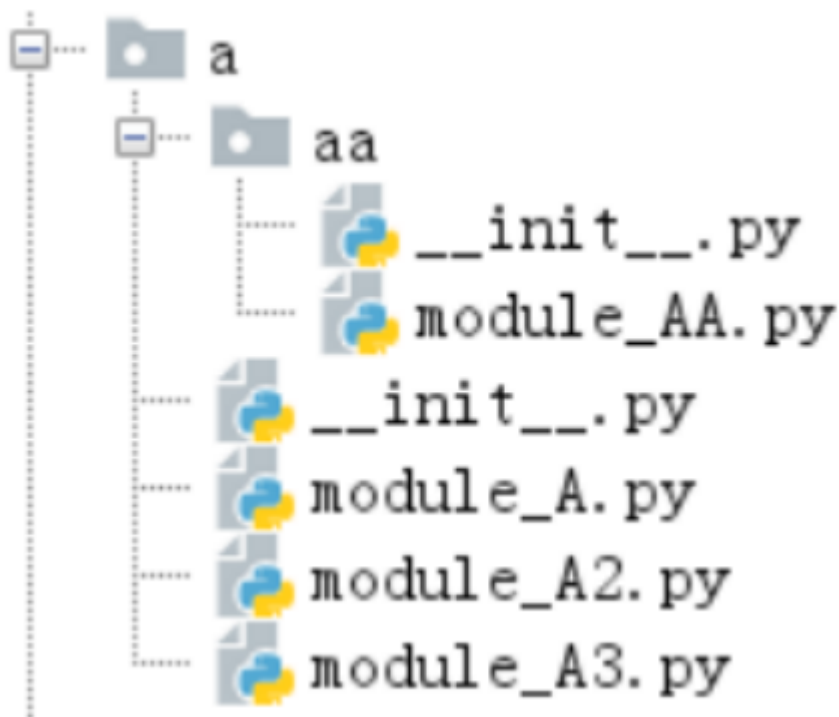
3.4.1 包的概念和结构

当一个项目中有很多个模块时，需要再进行组织。我们将功能类似的模块放到一起，形成了“包”。本质上，“包”就是一个必须有 `__init__.py` 的文件夹。典型结构如下：



包下面可以包含模块module，也可以再包含子包subpackage。

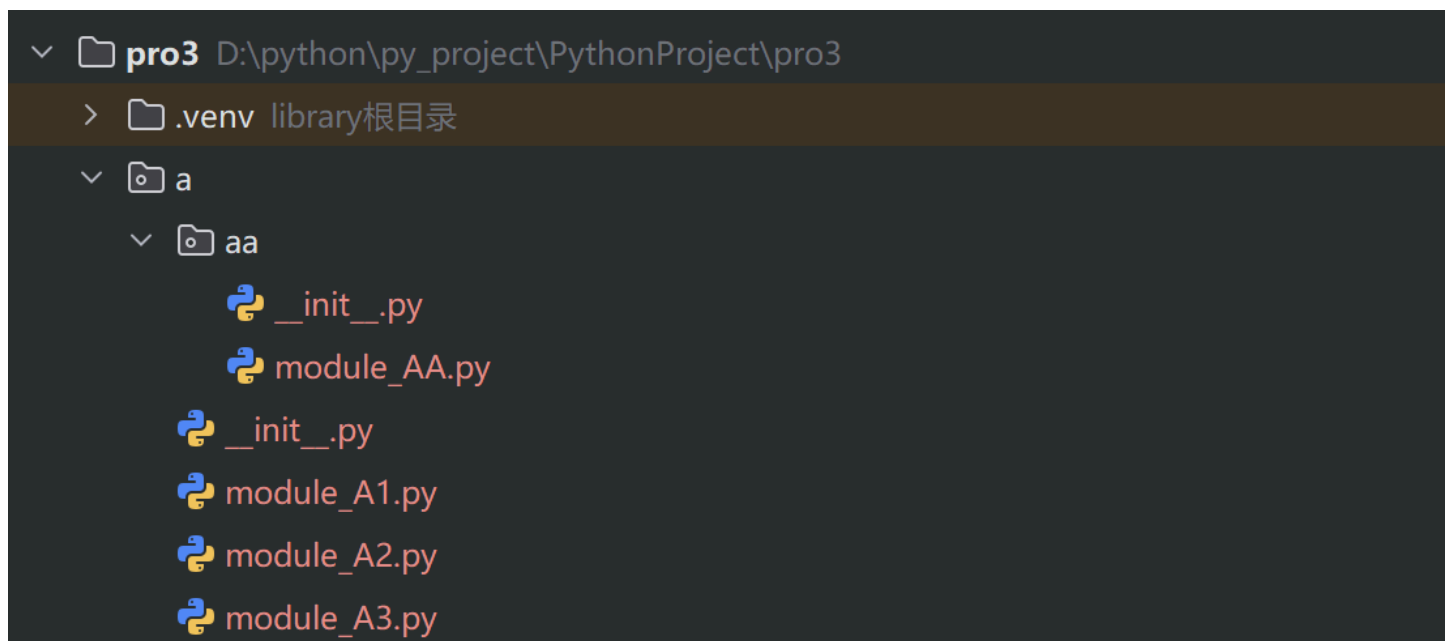
下图中，a 是上层包，下面有一个子包 aa。每个包里面都有 `__init__.py` 文件。



3.4.2 Pycharm 中创建包

在 pycharm 开发环境中创建包：右键项目 -> new -> python package 输入包名，就会创建这个包，`__init__.py` 文件会自动创建。

3.4.3 导入包操作和本质



1. `import a.aa.module_AA`

在使用时，必须加完整名称来引用，比如 `a.aa.module_AA.fun_AA()`

2. `from a.aa import module_AA`

在使用时，可以直接使用模块名，比如 `module_AA.fun_AA()`

3. `from a.aa.module_AA import fun_AA` 直接导入函数

在使用时，直接可以使用函数名。比如 `fun_AA()`



1. `from package import item` 这种语法中，item 可以是包、模块、，也可以是函数、类、变量
2. `import item1, item2` 这种语法中，item 必须是包或者模块，不能是其他

导入包的本质 其实是“导入了包的 `__init__.py`”文件。也就是说，`import pack1` 意味着执行了包 pack1 下面的 `__init__.py` 文件。这样，可以在 `__init__.py` 中批量导入需要的模块，而不再需要一个个导入。

3.4.4 `__init__.py` 的三个核心作用

1. 作为包的表示，不能删除。
 2. 导入包实质实质性 `__init__.py` 文件，可以在这个文件中做包的初始化，以及需要统一执行的代码、批量导入。
- 测试导入包的本质用法

The screenshot illustrates the execution of a Python package. The file explorer on the left shows a project structure with a package 'a' containing `__init__.py` and `module_b1.py`. The `__init__.py` file (highlighted with a red arrow) contains the following code:

```
1 import math
2 import turtle
3
4 print('a 包')
```

The `module_b1.py` file (highlighted with a red arrow) contains the following code:

```
1 import a
2
3 if __name__ == '__main__':
4     print(a.math.sin(a.math.pi / 2))
```

The console output at the bottom shows the execution path and the results:

```
D:\python\py_project\PythonProject\pro3\.venv\Scripts\python.exe D:\python\py_project\PythonProject\pro3\
a 包
1.0
```

3.4.5 用 `*` 导入包

`import *` 这种语句理论上是希望文件系统找出包中所有的子模块，然后导入它们。这可能会花很多时间。python 解决方案是提供一个明确的 `包索引`。

这个索引由 `__init__.py` 定义 `__all__` 变量，该变量是一个 `列表`，其中可以定义：
`__all__ = ['module_A1', 'module_A2']`

这意味着，`from b import *` 会从对应的包中导入以上两个子模块。但仍然不建议这样导入。

3.5 PIP 模块管理工具

pip 是一个现代的，通用的Python包管理工具。提供了对 Python 包的查找、下载、安装、卸载的功能。

3.5.1 第一种方式：命令行下远程安装

1. pip更换数据源(由于访问国外网站慢，建议更换)：

家目录中，创建 pip 目录，然后增加文件：pip.ini 内容拷贝下面的即可(不要加其他字符)：

代码块

```
1  [global]
2  index-url = https://mirrors.aliyun.com/pypi/simple/
3
4  [install]
5  trusted-host=mirrors.aliyun.com
```

Linux的家目录：~ 增加目录和文件：`~/.pip/pip.conf`

Windows的家目录是：`c:/user/用户名` 增加目录和文件：`c:/user/用户名/pip/pip.ini`

临时使用镜像源，不改全局配置：`pip install -i`
`http://mirrors.aliyun.com/pypi/simple pandas`

2. 其它数据源

阿里云 <http://mirrors.aliyun.com/pypi/simple/>

豆瓣：<http://pypi.douban.com/simple/>

中国科学技术大学：<https://pypi.mirrors.ustc.edu.cn/simple>

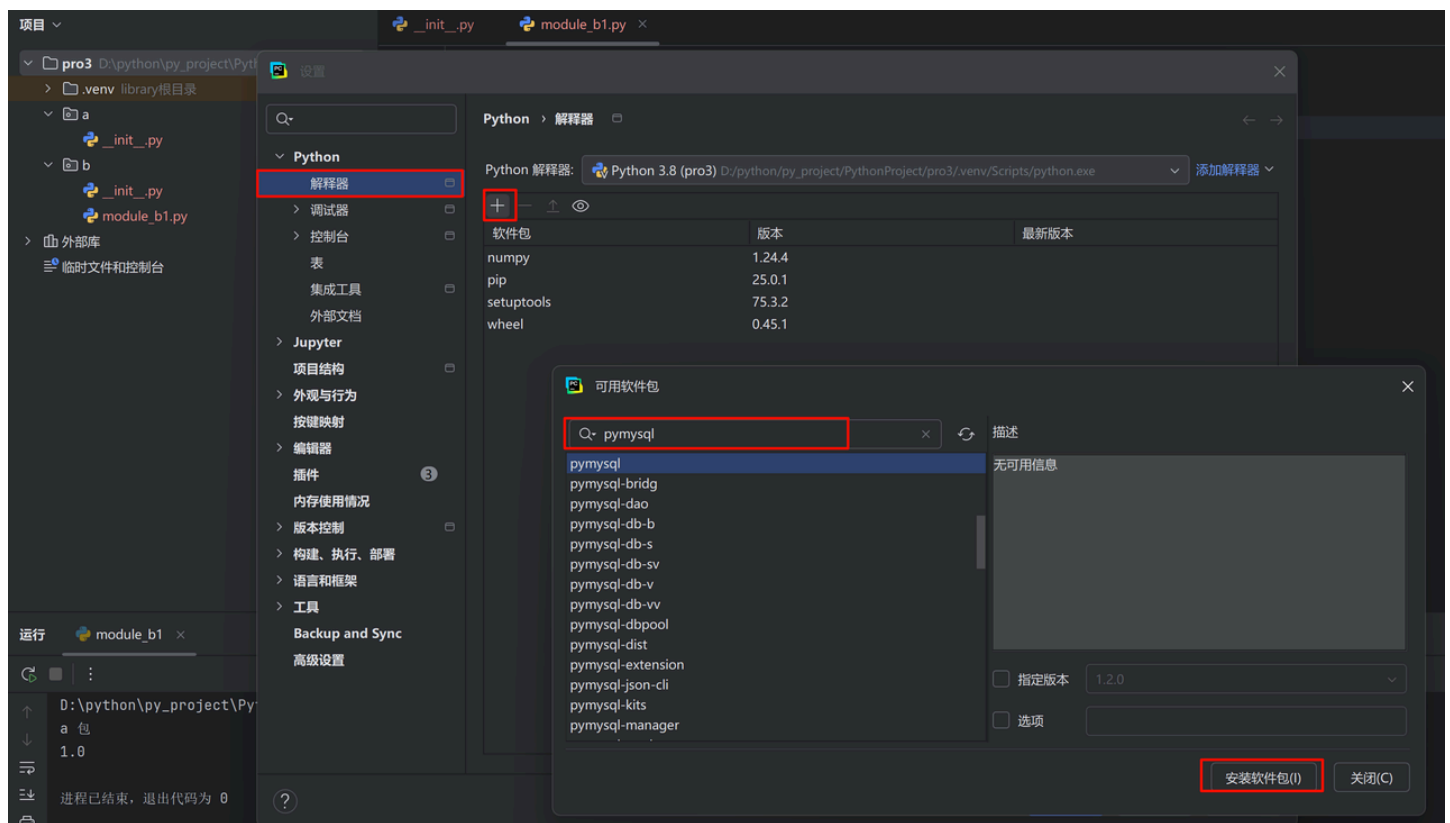
清华：<https://pypi.tuna.tsinghua.edu.cn/simple>

华中理工大学：<http://pypi.hustunique.com/simple>

山东理工大学：<http://pypi.sdutlinux.org/simple>

V2EX：<http://pypi.v2ex.com/simple>

3.5.2 第二种方式：pycharm 中直接安装到项目



4. Python 项目管理

4.1 Anaconda 和 Miniconda 介绍

Anaconda指的是一个**非常强大的Python发行版本**，其包含了 conda、Python等180多个科学包及其依赖项。因为包含了大量的 科学包，Anaconda 的下载文件比较大，如果只需要某些包，或者需要节省带宽或存储空间，也可以使用Miniconda这个较小的发行版（仅包含conda和 Python）。

conda是一个开源的包、环境管理器，可以用于**在同一个机器上安装不同版本的软件包及其依赖**，并能够在不同的环境之间切换。

Miniconda下载地址

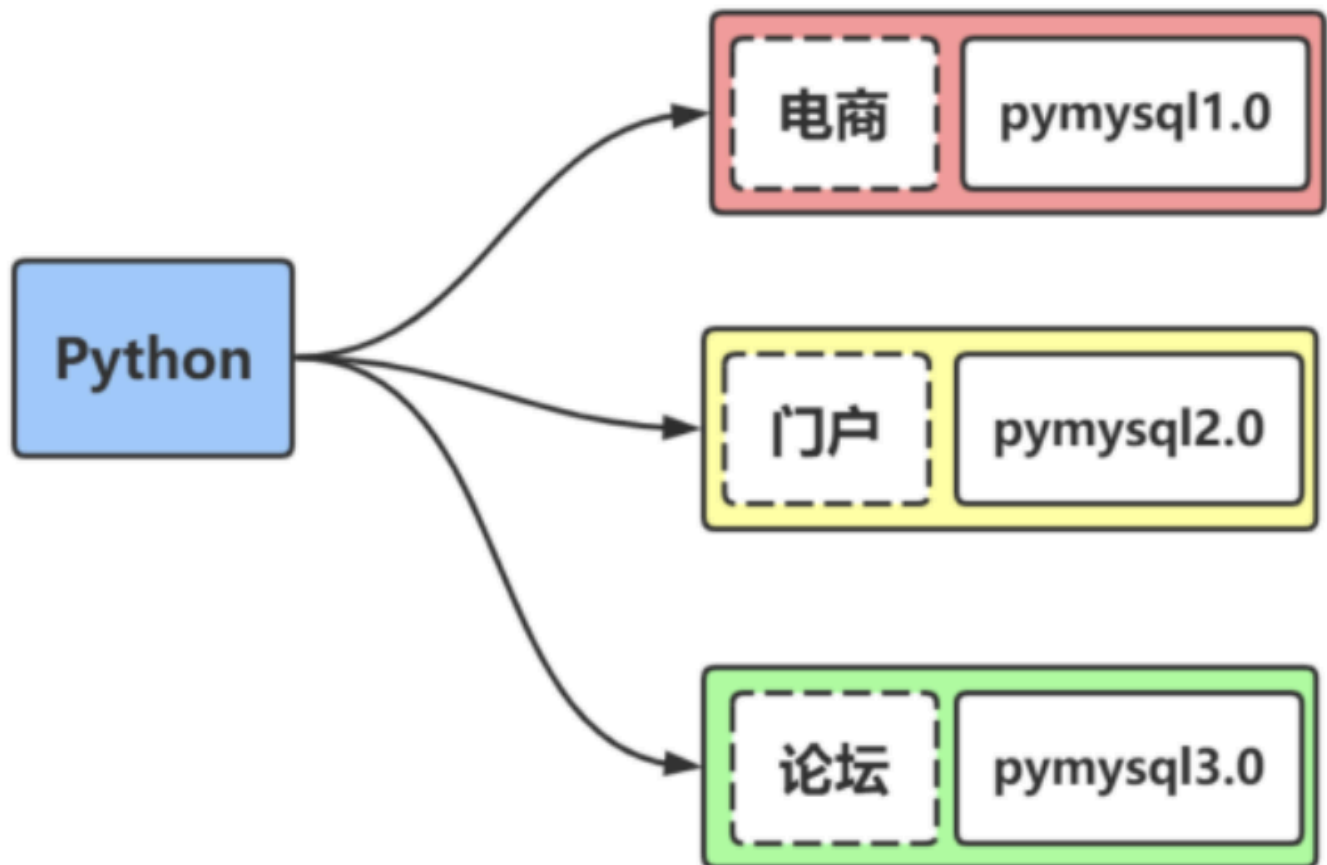
<https://repo.anaconda.com/miniconda/>

<https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/>

4.2 conda 虚拟环境

Python 应用经常需要使用一些第三方包或者模块，有时需要依赖特定的包或者库的版本，所以**不能有一个能适应所有 Python 应用的软件环境**。解决这一问题的方法就是**虚拟环境**。

虚拟环境是一个包含了特定 Python 解析器以及一些软件包，不同的应用程序可以使用不同的虚拟环境，从而解决了依赖冲突问题，而且虚拟环境中只需要安装应用相关的包或者模块，可以给部署提供便利。



4.2.1 Anaconda 虚拟环境命令

代码块

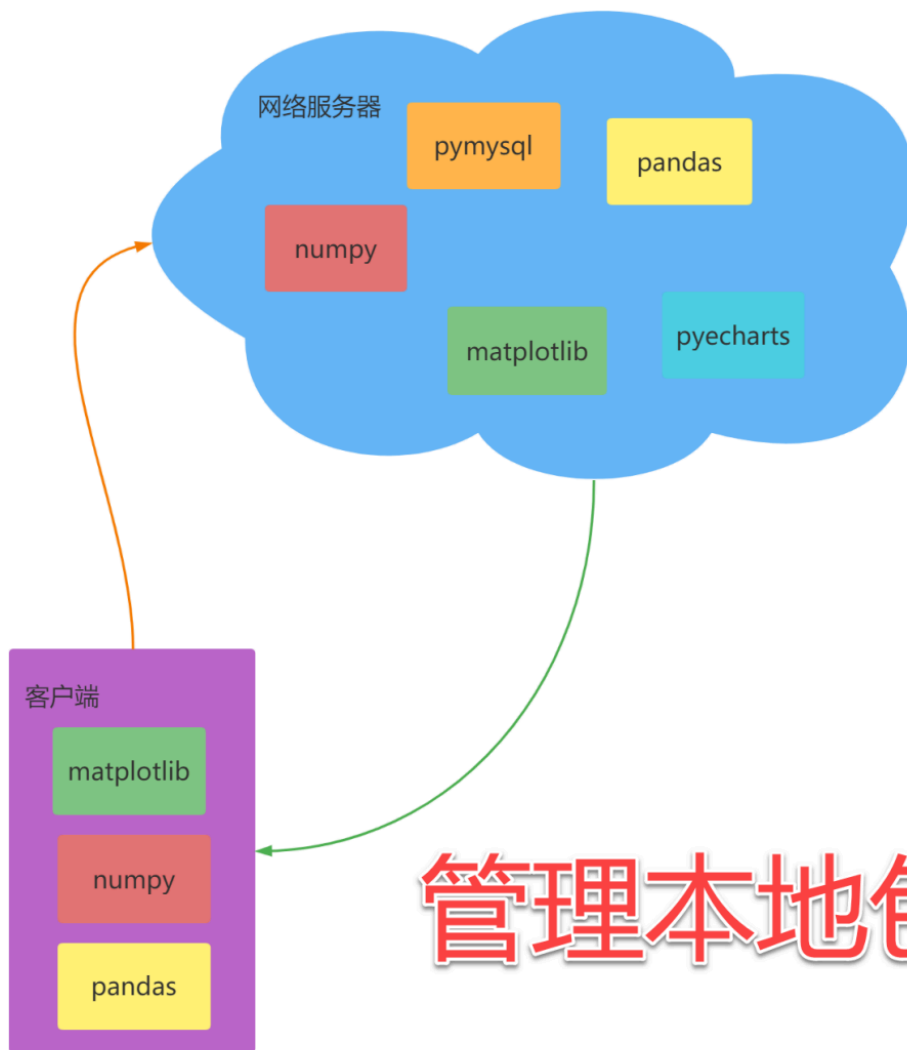
```
1  # 1、显示已存在的虚拟环境，有 * 号就表示当前在这个环境
2  conda info --envs
3
4  # 2、创建虚拟环境
5  conda create -n 虚拟环境名称 python=版本号
6
7  # 3、删除虚拟环境
8  conda remove -n 虚拟环境名称 --all
9
10 # 4、激活\进入虚拟环境
11 conda activate 虚拟环境名称
12
13 # 5、退出虚拟环境
14 conda deactivate 虚拟环境名称
15
16 # 6、列出当前虚拟环境中的所有包及其版本
17 conda list
```

```
(tf) PS C:\Users\huyuanhai> conda info --envs
# conda environments:
#
base                C:\Users\huyuanhai\anaconda3
tf                  * C:\Users\huyuanhai\anaconda3\envs\tf
```

```
(tf) PS C:\Users\huyuanhai> conda list
# packages in environment at C:\Users\huyuanhai\anaconda3\envs\tf:
#
# Name                                Version                                Build      Channel
blas                                  1.0                                    mkl
bzip2                                1.0.8                                h2bbff1b_6
ca-certificates                      2026.7.16                            haa95532_0
intel-openmp                         2025.0.0                            haa95532_1165
libexpat                             2.8.2                                hd7fb8db_1
libffi                               3.4.8                                h42d73b9_3
libzlib                              1.3.2                                h1c6eee0_0
mkl                                  2025.0.0                            h585ebfc_931
mkl-service                          2.7.2                                py312h1b35211_0
mkl_fft                             2.2.0                                py312h18b77b3_0
mkl_random                          1.4.1                                py312hf6ff4c0_0
numpy                                2.5.1                                py312h9606935_0
numpy-base                          2.5.1                                py312h89dd80d_0
openssl                              3.5.7                                hbb43b14_0
```

创建虚拟环境后，默认会在Anaconda安装目录的 `envs` 下，创建虚拟环境相关文件。

4.3 conda 软件模块的管理



管理本地包的增删改

conda是一个软件的管理器，可用来下载、删除Python的软件包(与 pip 有些类似)。

代码块

```
1  # 1、安装模块、软件
2  conda install 软件/模块名
3
4  # 2、卸载模块、软件
5  conda remove 软件/模块
6
7  # 3、查看虚拟环境中的包
8  conda list
```

pip 管理

代码块

```
1  # 1、安装模块、软件
2  pip install 软件/模块名
3
4  # 2、卸载模块、软件
```

```
5  pip uninstall 软件/模块
6
7  # 3、查看虚拟环境中的包
8  pip list
```

conda 对比 pip

conda在安装包时会同时处理所有依赖关系，包括

- Python解释器版本
- C/C++库
- CUDA等硬件相关的依赖
- 其他编程语言的依赖

conda使用严格的依赖解析器，可以预先检查所有依赖冲突；

pip 只管理Python包，不管理系统级别的依赖。

4.4 数据源与路径设置

1、重点

- 设置虚拟环境目录（转移到其它位置，避免磁盘过载）
- 设置下载软件的数据源（国外的比较慢，可以设置成国内的，下载快）
- 设置缓存目录（不是每次都需要下载，会缓存一份，提高效率）

2、配置文件

Linux 用户可以通过修改用户目录下的 `.condarc` 文件。

Windows 用户无法直接创建名为 `.condarc` 的文件，可先执行 `conda config` 生成该文件之后再修改。

↻ 🖥️ > 此电脑 > 本地磁盘 (C:) > 用户 > huyuanhai >				
📁 📁 🔍 🔗 🗑️ ↑↓ 排序 ▾ ≡ 查看 ▾ ⋮				
名称	修改日期	类型	大小	
🎬 视频	2025/10/22 7:54	文件夹		
📁 搜索	2025/10/26 9:07	文件夹		
🖼️ 图片	2026/6/14 21:23	文件夹		
📄 文档	2026/7/28 22:58	文件夹		
⬇️ 下载	2026/8/4 8:39	文件夹		
🎵 音乐	2025/10/20 16:17	文件夹		
🖥️ 桌面	2026/8/2 10:01	文件夹		
📄 .claude.json	2026/7/12 0:15	JSON 源文件	1 KB	
📄 .condarc	2026/8/4 21:31	CONDARC 文件	1 KB	

3、conda信息

`conda info` 可以获取 conda 的系统信息。

代码块

```

1  (tf) PS C:\Users\huyuanhai> conda info
2
3      active environment : tf
4      active env location : C:\Users\huyuanhai\anaconda3\envs\tf
5      shell level        : 2
6      user config file   : C:\Users\huyuanhai\.condarc
7      populated config files : C:\Users\huyuanhai\.condarc
8      conda version      : 4.8.3
9      conda-build version : 3.18.11
10     python version     : 3.8.3.final.0
11     virtual packages   :
12     base environment   : C:\Users\huyuanhai\anaconda3 (writable)
13     channel URLs       : https://repo.anaconda.com/pkgs/main/win-64
14                        https://repo.anaconda.com/pkgs/main/noarch
15                        https://repo.anaconda.com/pkgs/r/win-64
16                        https://repo.anaconda.com/pkgs/r/noarch
17                        https://repo.anaconda.com/pkgs/msys2/win-64
18                        https://repo.anaconda.com/pkgs/msys2/noarch
19     package cache      : C:\Users\huyuanhai\anaconda3\pkgs
20                        C:\Users\huyuanhai\.conda\pkgs
21                        C:\Users\huyuanhai\AppData\Local\conda\conda\pkgs
22     envs directories   : C:\Users\huyuanhai\anaconda3\envs
23                        C:\Users\huyuanhai\.conda\envs
24                        C:\Users\huyuanhai\AppData\Local\conda\conda\envs

```

```
25         platform : win-64
26         user-agent : conda/4.8.3 requests/2.24.0 CPython/3.8.3 Windows/10
           Windows/10.0.26100
27         administrator : False
28         netrc file : None
29         offline mode : False
```

4.4.1 conda 数据源

以清华大学开源软件镜像站为例：<https://mirrors.tuna.tsinghua.edu.cn/help/anaconda/>

- 直接修改配置文件

The screenshot shows the '使用方法' (Usage) page for the TUNA Anaconda mirror. It explains that the mirror provides Anaconda channels and third-party sources (conda-forge, msys2, pytorch, etc.). It instructs users to modify the `.condarc` file in their user directory to use the mirror.

不同系统下的 `.condarc` 目录如下:

- Linux: `${HOME}/.condarc`
- macOS: `${HOME}/.condarc`
- Windows: `C:\Users\<YourUserName>\.condarc`

注:

- Windows 用户无法直接创建名为 `.condarc` 的文件, 可先执行 `conda config --set show_channel_urls yes` 生成该文件之后再修改。
- 由于更新过快难以同步, TUNA 等镜像站不同步 `pytorch-nightly`, `pytorch-nightly-cpu`, `ignite-nightly` 这三个包。
- 如果您正在从某一镜像源切换到另一镜像源, 请检查镜像源是否同步了您所需要的 repo, 以及该 repo 是否支持您使用的平台 (e.g. linux-64)。
- 为了保证以下配置在所有镜像站可用, 配置中只加入了少量必须的第三方源, 您可以在下方的列表中自行寻找并添加其他第三方源。

配置示例 (复制到配置文件中):

```
channels:
  - defaults
show_channel_urls: true
default_channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2
custom_channels:
  conda-forge: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
```

即可添加 Anaconda Python 免费仓库。

需要看 `conda info` 信息中, `populated config files` 有多个配置文件, 需要改最近的一个。修改后, 使用 `conda info` 查看如下:

```
(tf) PS C:\Users\huyuanhai> conda info
```

```
active environment : tf
active env location : C:\Users\huyuanhai\anaconda3\envs\tf
shell level : 2
user config file : C:\Users\huyuanhai\.condarc
populated config files : C:\Users\huyuanhai\.condarc
conda version : 4.8.3
conda-build version : 3.18.11
python version : 3.8.3.final.0
virtual packages :
base environment : C:\Users\huyuanhai\anaconda3 (writable)
channel URLs : https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/win-64
                https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/noarch
                https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r/win-64
                https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r/noarch
                https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2/win-64
                https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2/noarch
package cache : C:\Users\huyuanhai\anaconda3\pkgs
                 C:\Users\huyuanhai\.conda\pkgs
                 C:\Users\huyuanhai\AppData\Local\conda\conda\pkgs
envs directories : C:\Users\huyuanhai\anaconda3\envs
                  C:\Users\huyuanhai\.conda\envs
                  C:\Users\huyuanhai\AppData\Local\conda\conda\envs
platform : win-64
user-agent : conda/4.8.3 requests/2.24.0 CPython/3.8.3 Windows/10 Windows/10.0.26100
administrator : False
netrc file : None
offline mode : False
```

- 通过命令修改

代码块

```
1 conda config --add channels
  https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
2 conda config --add channels
  https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/
3 conda config --add channels
  https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
4 conda config --add channels
  https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/msys2/
```

- pip数据源修改

代码块

```
1 pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

4.4.2 设置虚拟环境目录

在上述配置文件中，继续增加这个配置，修改后，在 `conda info` 中可以查看，对应 `envs directories` 这个字段。

代码块

```
1 envs_dirs:
```

4.4.3 设置缓存软件目录

在上述配置文件中，继续增加这个配置，修改后，在 `conda info` 中可以查看，对应 `package cache` 这个字段。

代码块

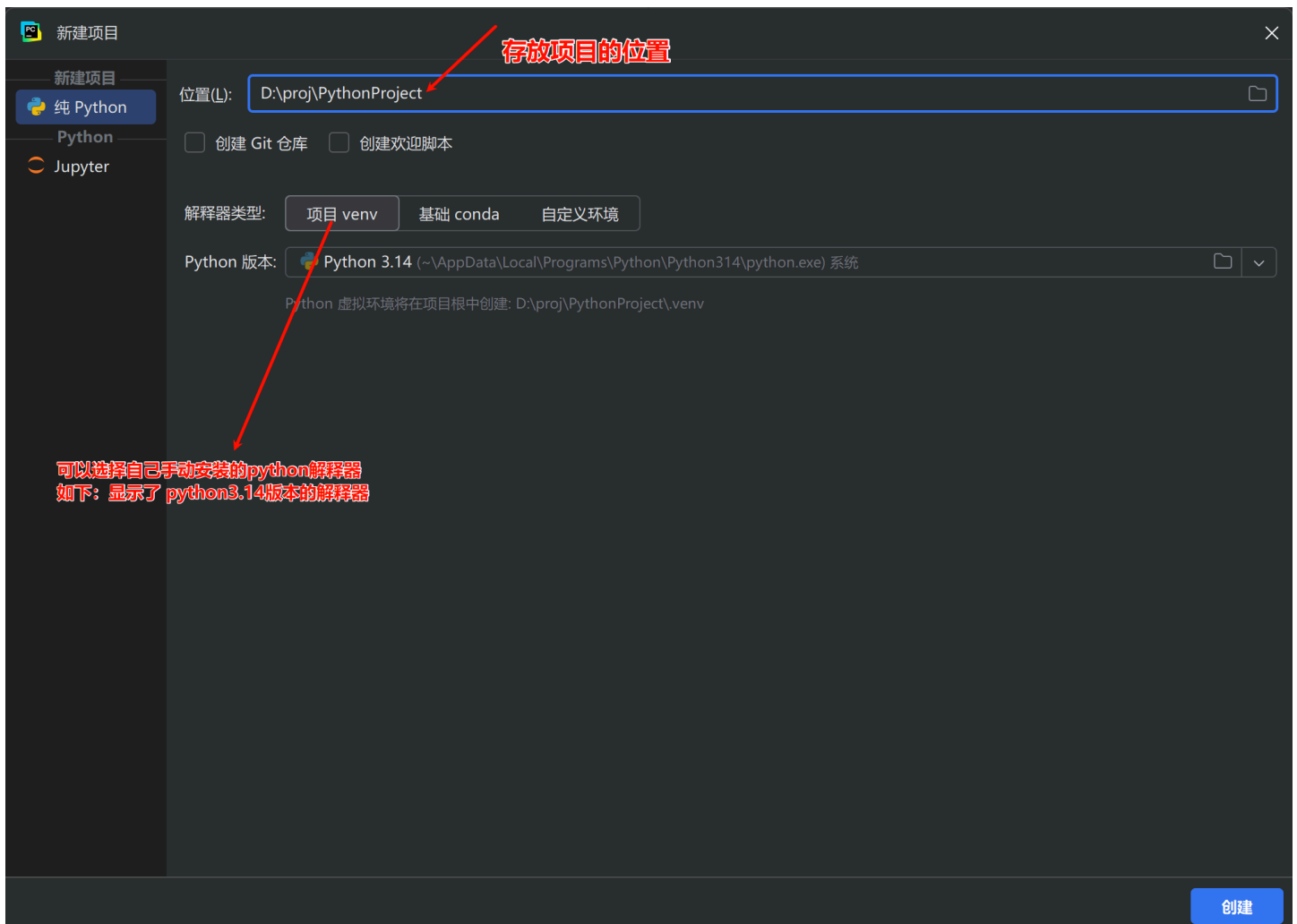
```
1 pkgs_dirs:
2   - D:\miniconda\pkgs
```

4.5 pycharm安装与使用

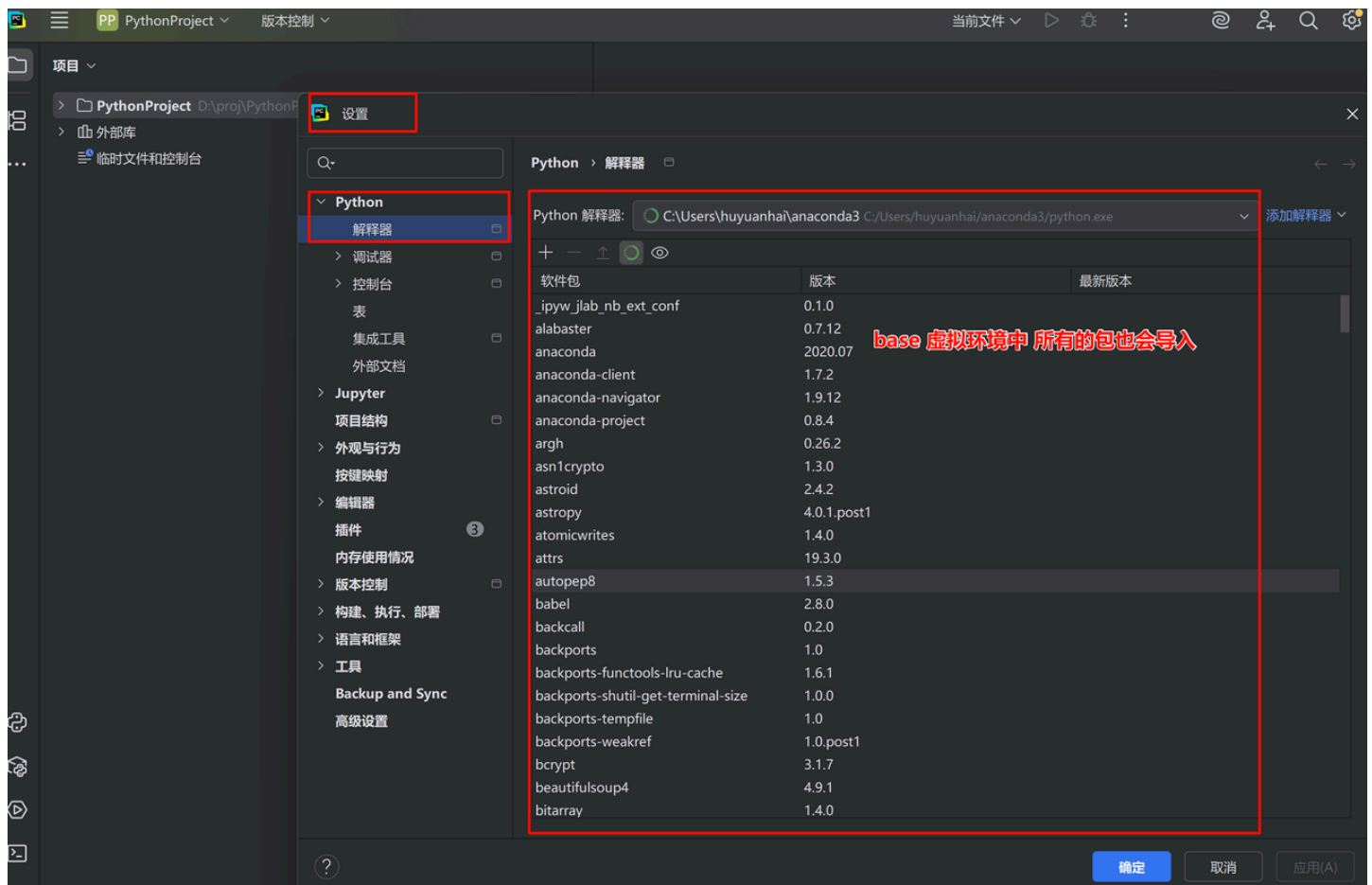
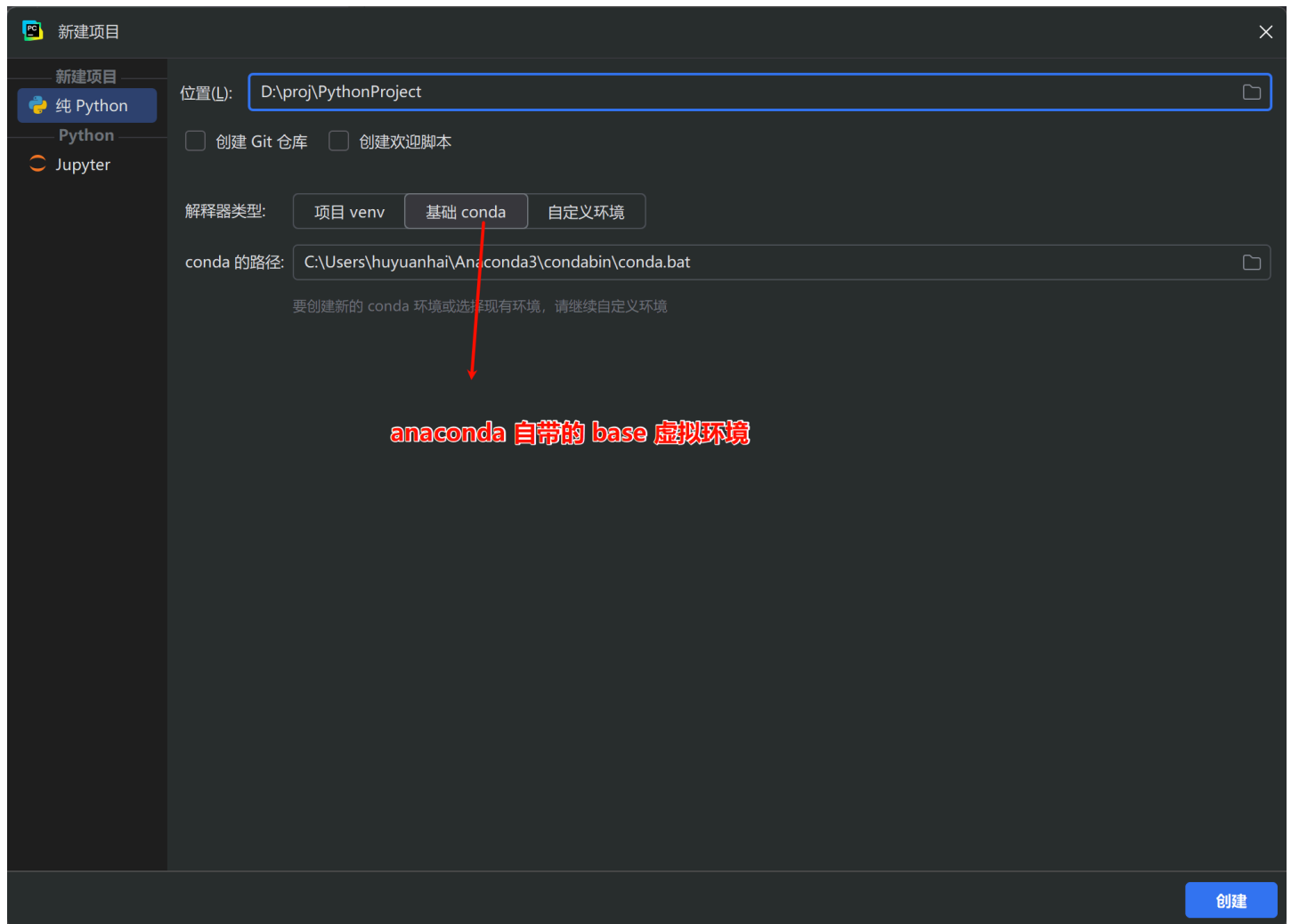
pycharm的下载 <https://www.jetbrains.com/pycharm/download>

pycharm项目虚拟环境

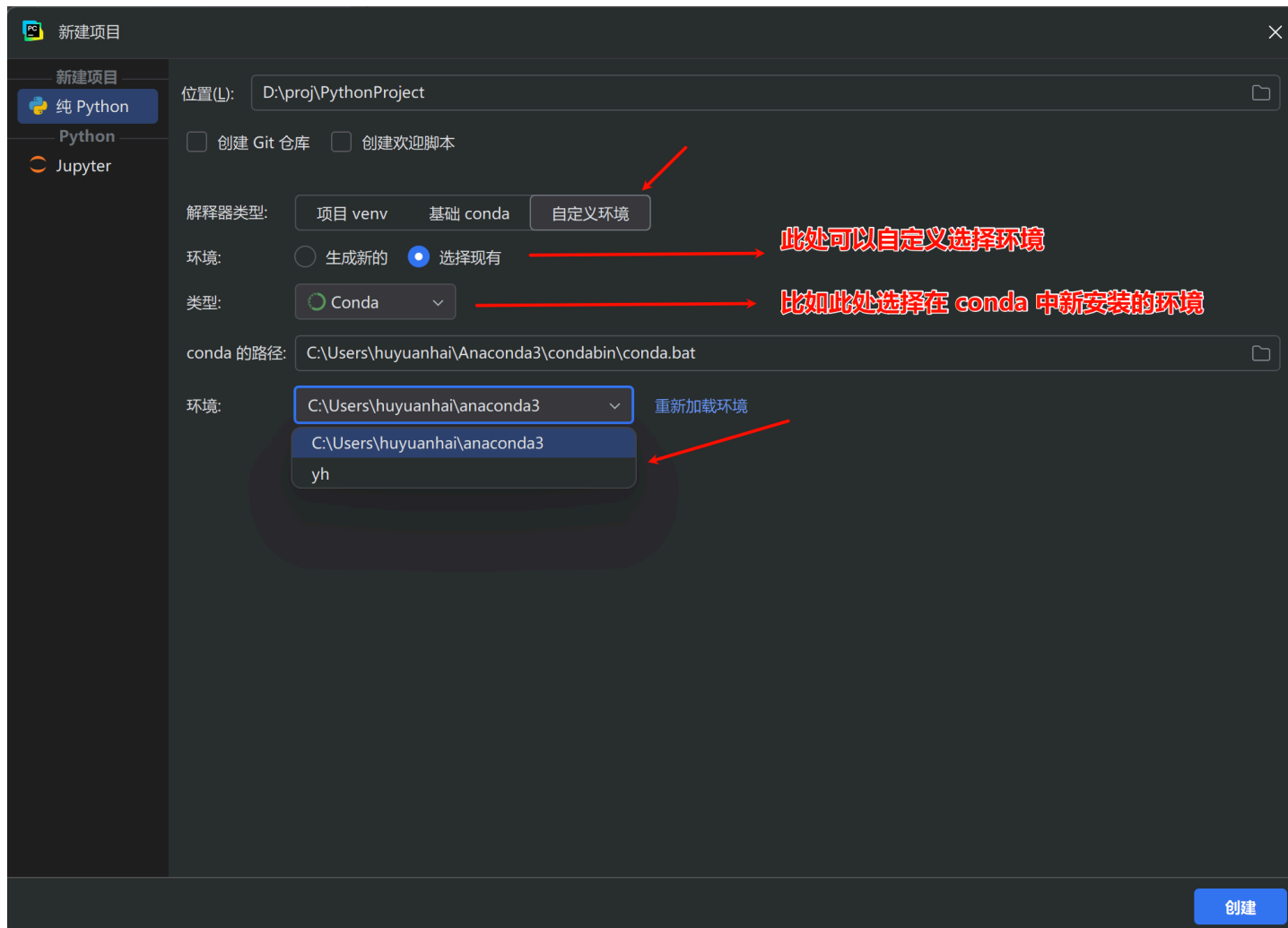
- 手动安装的python环境



- Anaconda base 虚拟环境

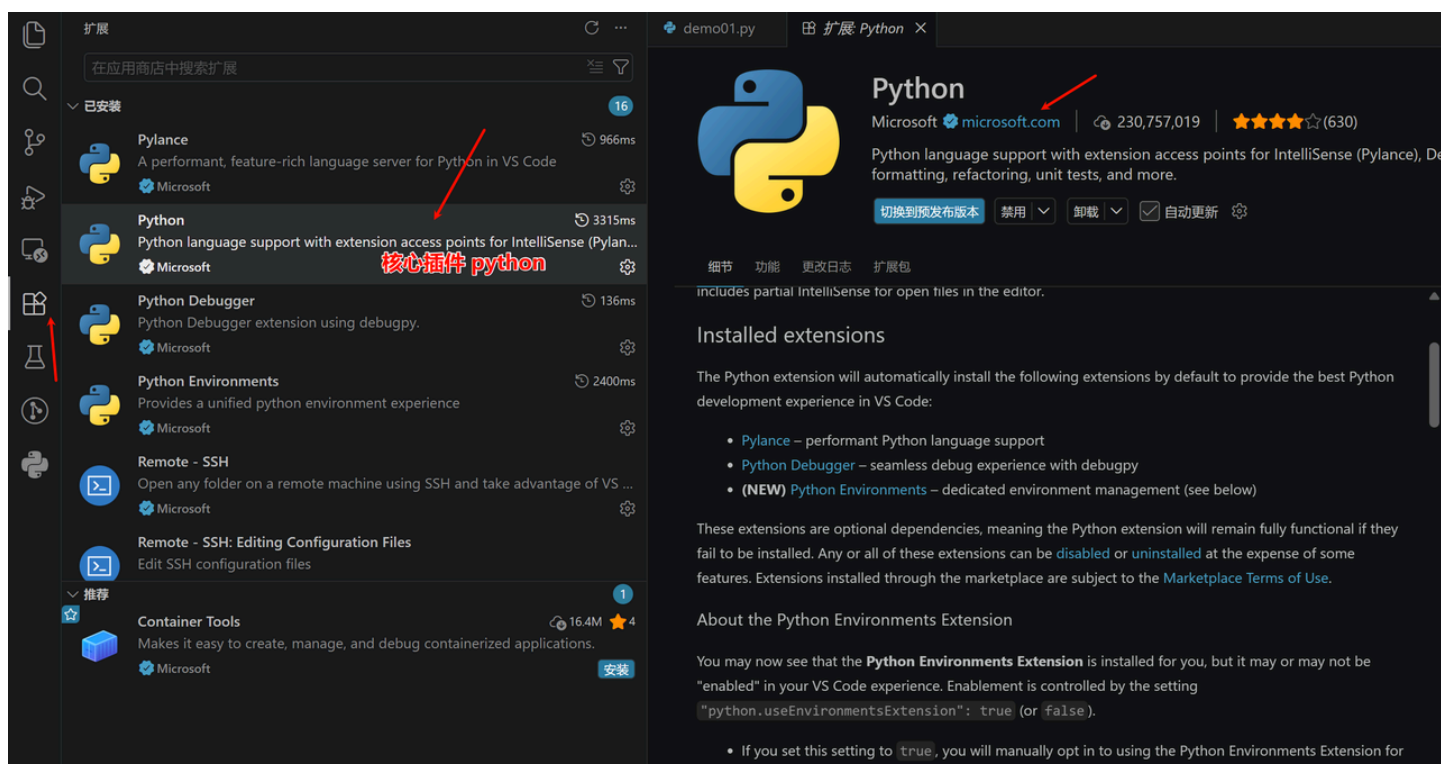


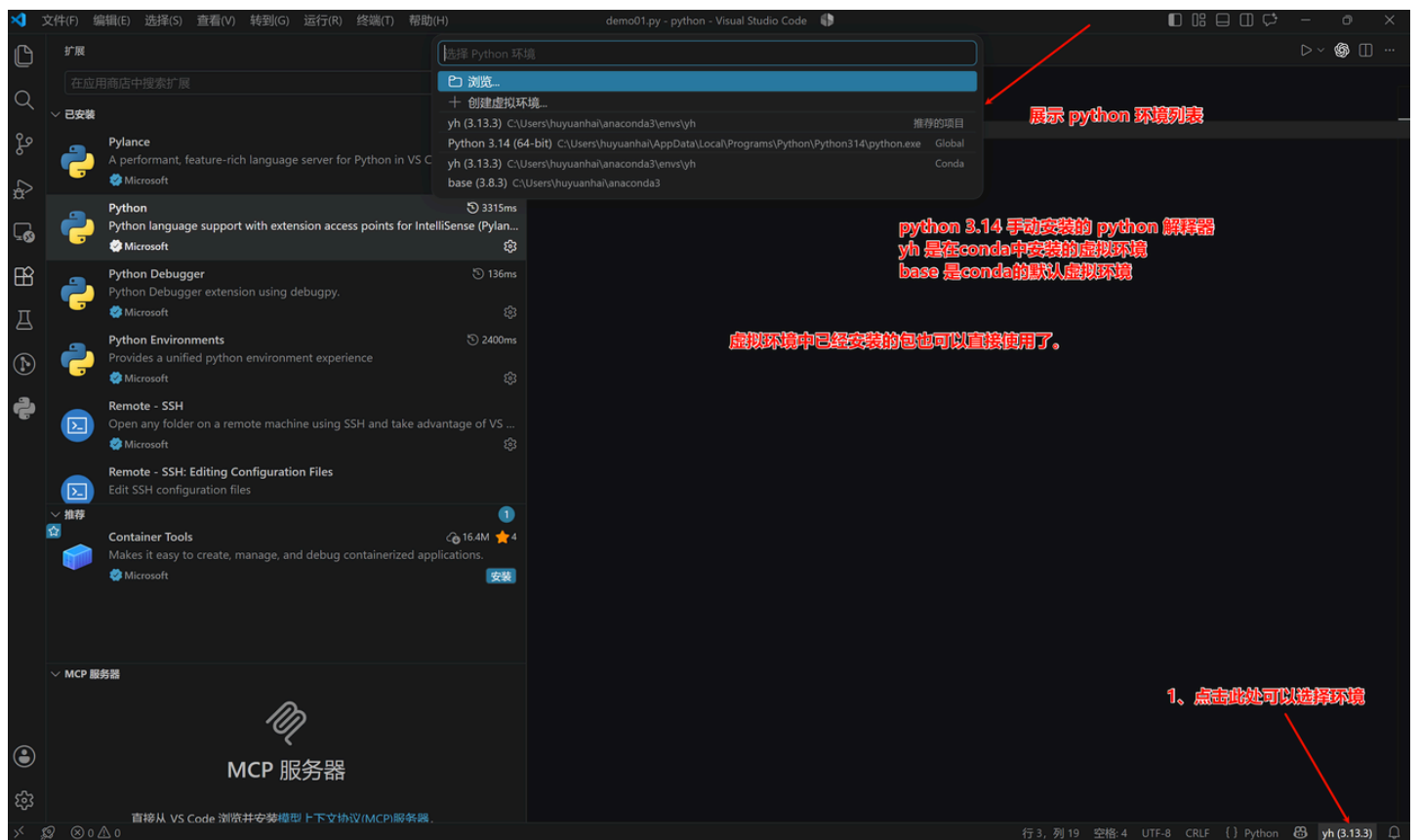
- 自定义选择python环境或虚拟环境



4.6 VSCode配置python环境

Vscode 简洁，需要安装插件来配置开发环境。





4.7 python项目管理文件

项目代码移植到其它主机或者平台，无法直接运行，需要有相同的解释器以及依赖包。如果在新的主机上逐个按版本安装，非常繁琐。

开发项目的时候 **不能直接无脑的在环境中安装依赖包**，否则要打包代码给到其它地方会导致不知道具体需要依赖哪些以及哪些版本的第三方库。需要用这个项目管理文件处理这个问题。

4.7.1 早期管理工具（使用 conda 虚拟环境测试）

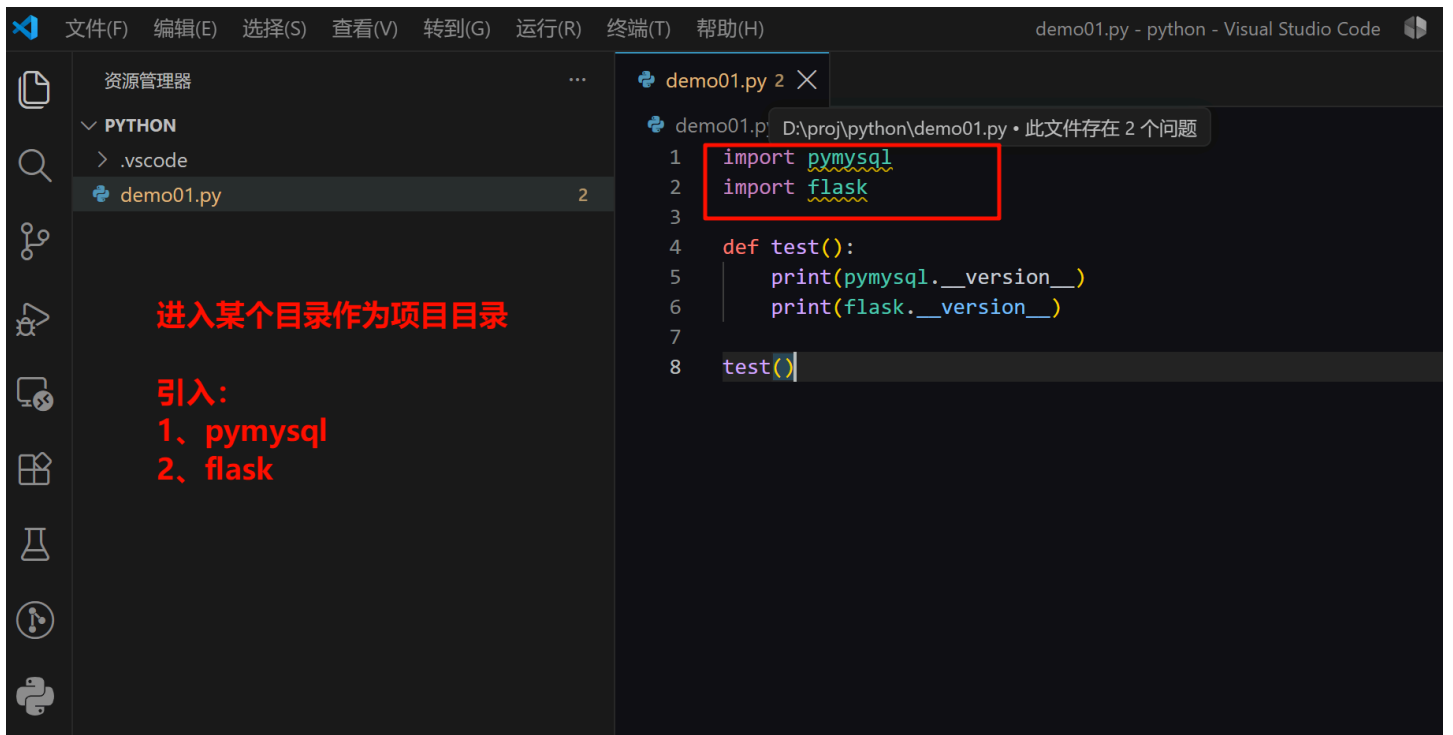
使用 **requirements.txt** 管理环境。

1. 使用 conda 创建虚拟环境测试

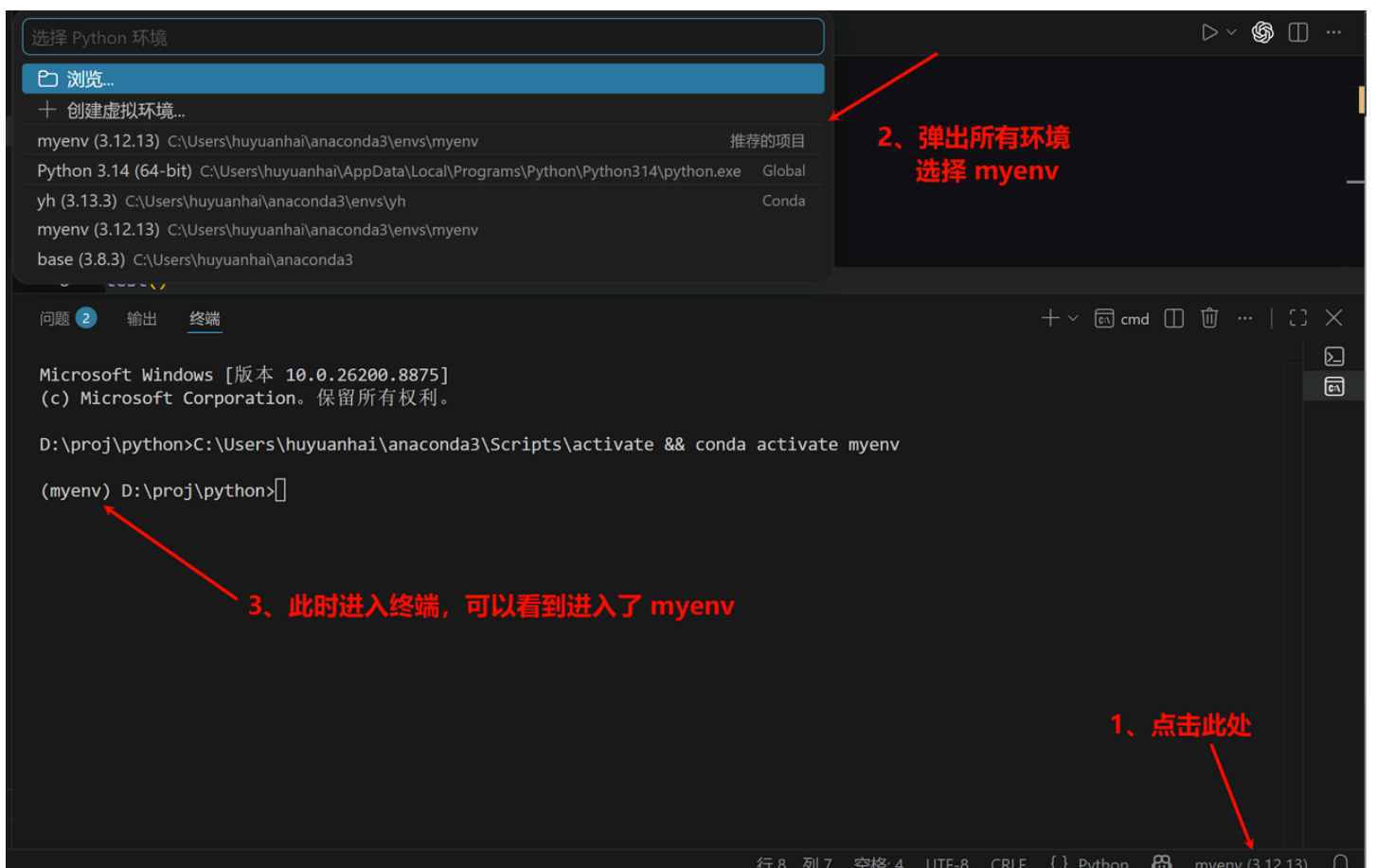
代码块

```
1 conda create -n myenv python=3.12
```

2. 使用 Vscode 进入某个目录作为项目目录



3. 指定上述创建的 myenv 虚拟环境作为这个项目的环境



4. 此时在这个虚拟环境中安装上述的两个模块 `pymysql` 和 `flask`

代码块

```
1 conda install pymysql
2
```



```
3 conda install flask
```

demo01.py X

demo01.py > ...

```
1 import pymysql
2 import flask
3
4 def test():
5     print(pymysql.__version__)
6     print(flask.__version__)
7
8 test()
```

问题 输出 终端

```
(myenv) D:\proj\python>conda list
# packages in environment at C:\Users\huyuanhai\anaconda3\envs\myenv:
#
# Name                          Version          Build       Channel
blinker                         1.9.0            py312haa95532_1 defaults
bzip2                           1.0.8            h2bbff1b_6  defaults
ca-certificates                 2026.7.16        haa95532_0  defaults
click                           8.4.1            py312haa95532_0 defaults
colorama                        0.4.6            py312haa95532_0 defaults
flask                           3.1.3            py312haa95532_0 defaults
itsdangerous                    2.2.0            py312haa95532_0 defaults
jinja2                          3.1.6            py312haa95532_0 defaults
libexpat                        2.8.2            hd7fb8db_1  defaults
libffi                           3.4.8            h42d73b9_3  defaults
libzlib                         1.3.2            h1c6eee0_0  defaults
markupsafe                      3.0.3            py312h1c6eee0_0 defaults
openssl                         3.5.7            hbb43b14_0  defaults
packaging                       26.1             py312haa95532_0 defaults
pip                             26.1.2           pyhc872135_0 defaults
pymysql                         1.2.0            py312haa95532_0 defaults
python                          3.12.13          hd7b1df3_3  defaults
setuptools                      83.0.0           py312haa95532_0 defaults
sqlite                          3.53.2           hee5a0db_0  defaults
tk                              8.6.15           hf199647_0  defaults
tzdata                          2026c            he532380_0  defaults
ucrt                            10.0.22621.0     haa95532_0  defaults
vc                              14.3             h2df5915_12 defaults
vc14_runtime                    14.44.35208      h4927774_12 defaults
vs2015_runtime                  14.44.35208      ha6b5a95_12 defaults
werkzeug                        3.1.6            py312haa95532_0 defaults
wheel                           0.47.0           py312haa95532_0 defaults
xz                              5.8.2            h53af0af_0  defaults
zlib                            1.3.2            h1c6eee0_0  defaults
```

5. 此时可以使用这两个模块

demo01.py X

demo01.py > ...

```
1  import pymysql
2  import flask
3
4  def test():
5      print(pymysql.__version__)
6      print(flask.__version__)
7
8  test()
```

问题

输出

终端

(myenv) D:\proj\python>python demo01.py

2.2.8

D:\proj\python\demo01.py:6: DeprecationWarning:
tion or 'importlib.metadata.version("flask")' in
print(flask.__version__)

3.1.3

(myenv) D:\proj\python>

6. `pip freeze` 命令生成 `requirements.txt` 管理文件

`pip freeze` 命令以特定的格式（包名==版本号）列出当前激活的 Python 环境中所有已安装的第三方包。

代码块

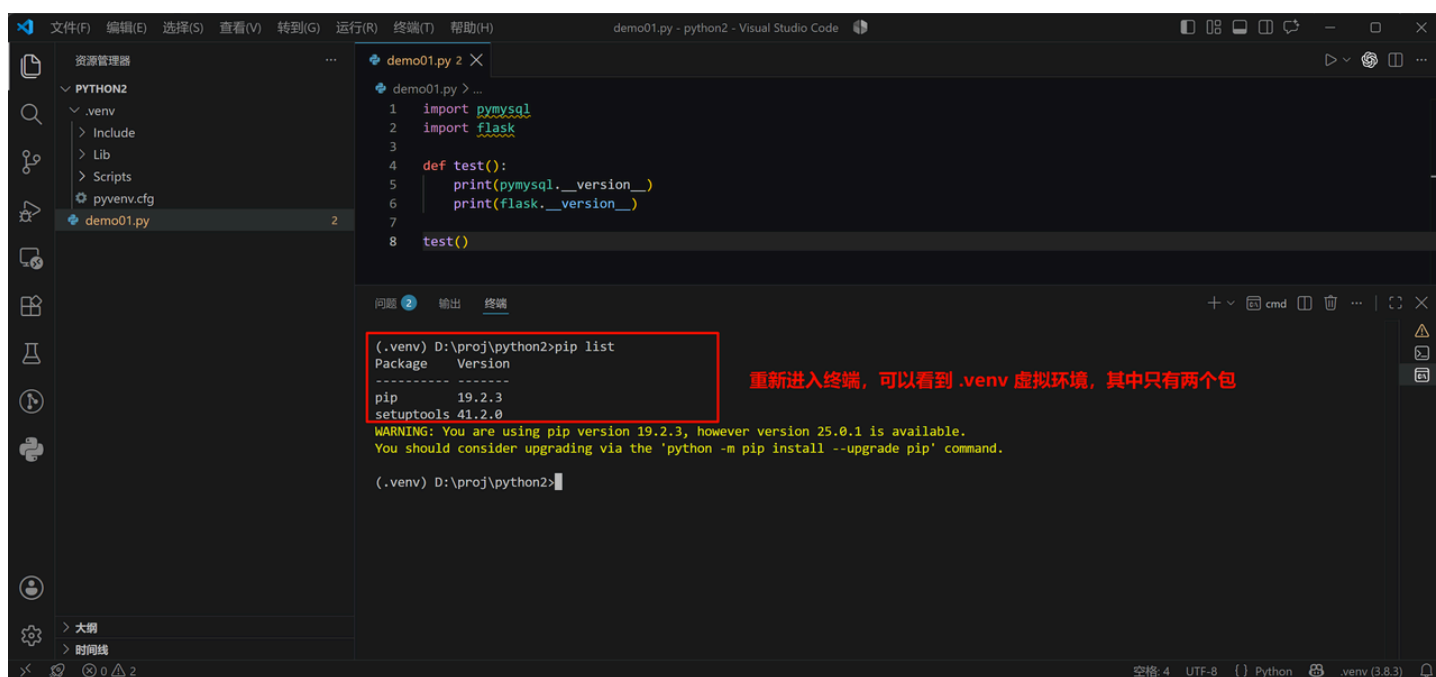
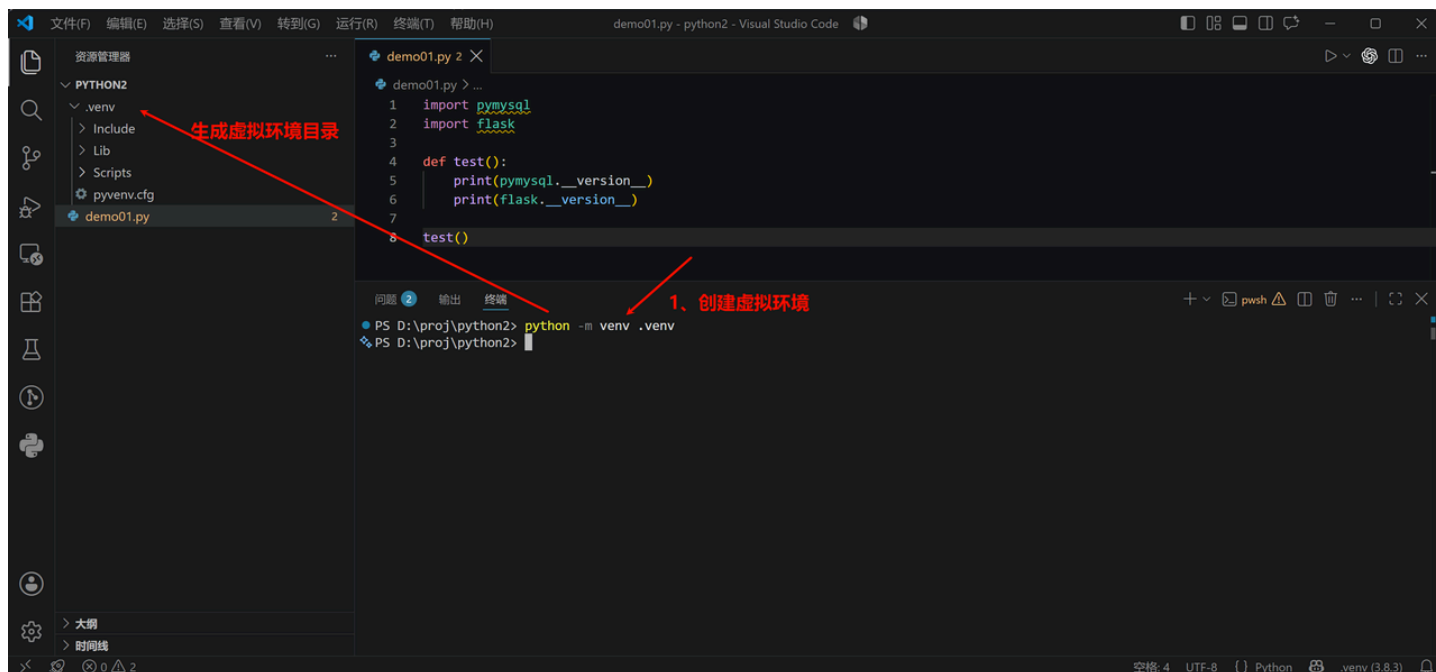
```
1 (myenv) D:\proj\python>pip freeze > requirements.txt
```

4.7.2 早期管理工具（使用 python 自带的虚拟环境测试）

1. 创建虚拟环境

代码块

```
1 python -m venv .venv
```



2. 安装 pymysql 和 flask 模块

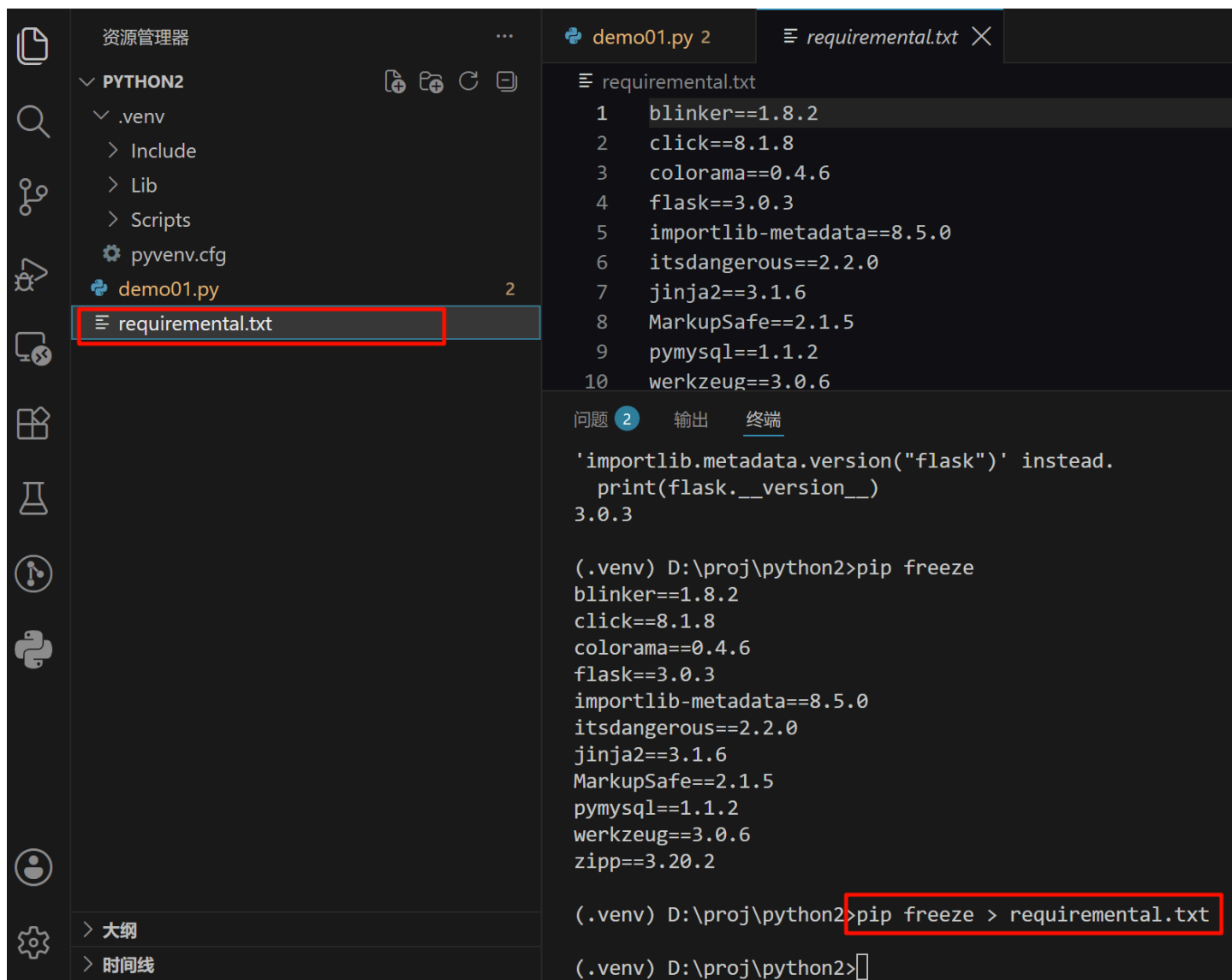
代码块

```
1 (.venv) D:\proj\python2> pip install pymysql flask # 相关依赖也会安装
2
3 (.venv) D:\proj\python2>pip list
4 Package              Version
5 -----
6 blinker               1.8.2
7 click                 8.1.8
8 colorama              0.4.6
9 flask                 3.0.3
10 importlib-metadata 8.5.0
11 itsdangerous          2.2.0
12 jinja2                3.1.6
13 MarkupSafe            2.1.5
14 pip                   19.2.3
15 pymysql               1.1.2
16 setuptools            41.2.0
17 werkzeug              3.0.6
18 zipp                  3.20.2
```

3. `pip freeze` 命令

代码块

```
1 (.venv) D:\proj\python2>pip freeze
2 blinker==1.8.2
3 click==8.1.8
4 colorama==0.4.6
5 flask==3.0.3
6 importlib-metadata==8.5.0
7 itsdangerous==2.2.0
8 jinja2==3.1.6
9 MarkupSafe==2.1.5
10 pymysql==1.1.2
11 werkzeug==3.0.6
12 zipp==3.20.2
```

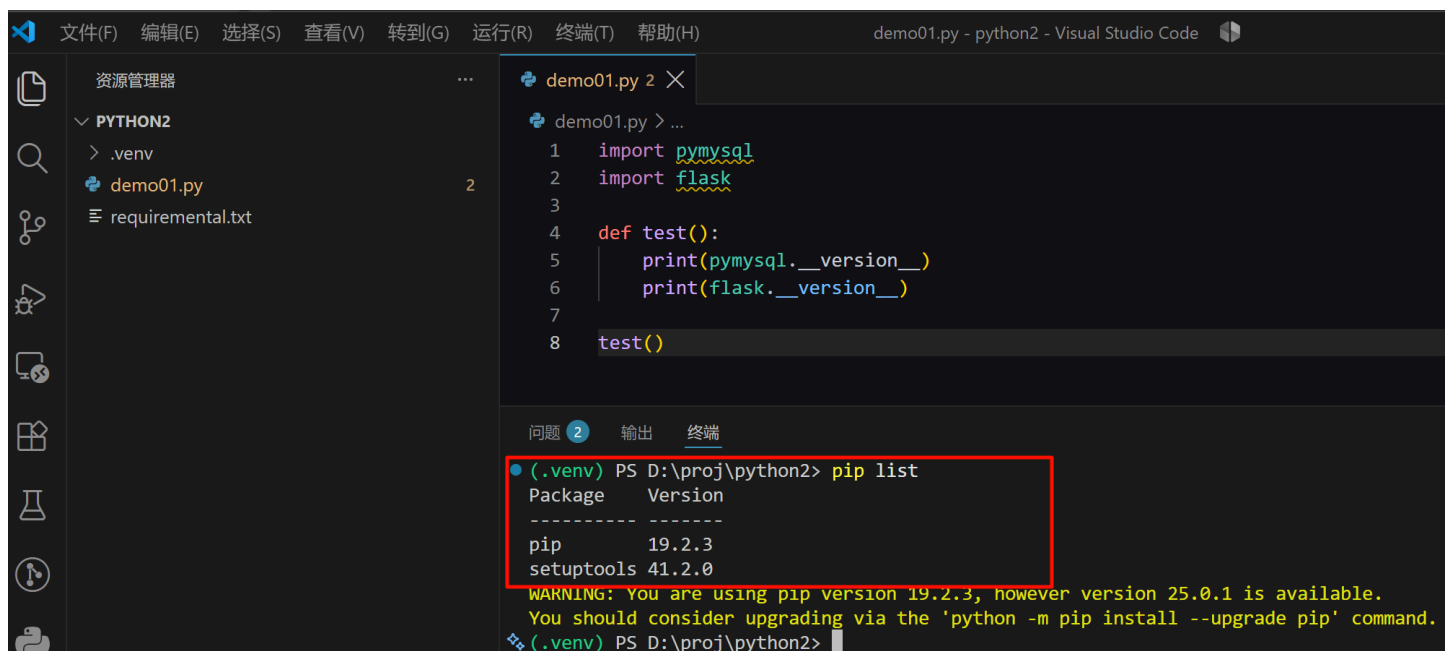
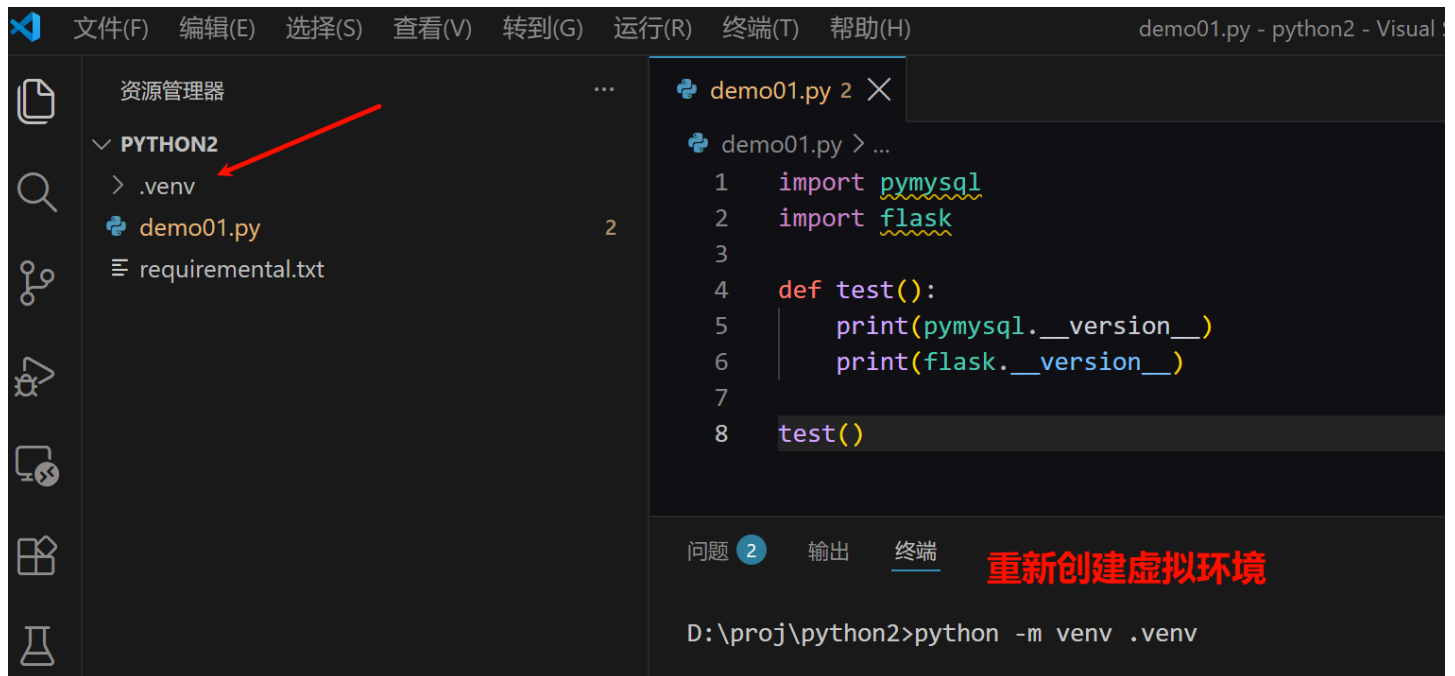


4. 移植项目到其它地方

把源码和刚才的 `requiremental.txt` 文件放到新的目录中



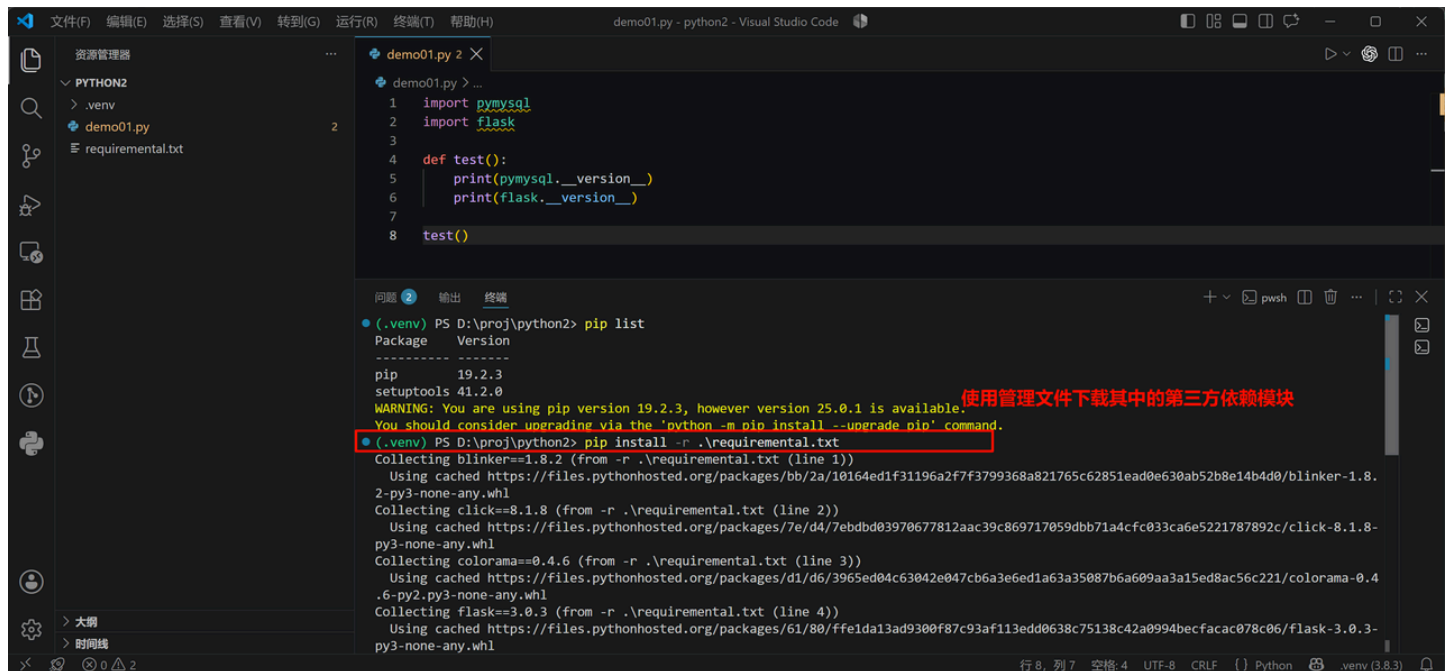
5. 重新搭建虚拟环境



6. 使用 `requiremental.txt` 安装依赖包

代码块

```
1 pip install -r requiremental.txt
```



4.7.3 进阶管理

早期管理的方式会把核心模块以及它本身依赖的模块全部展示出来，即使后面卸载了核心模块，其依赖的模块还会保留，导致“幽灵依赖”。

进阶管理就是解决了这个问题。它**显式声明需要哪些核心模块**，可以集中管理其它工具。

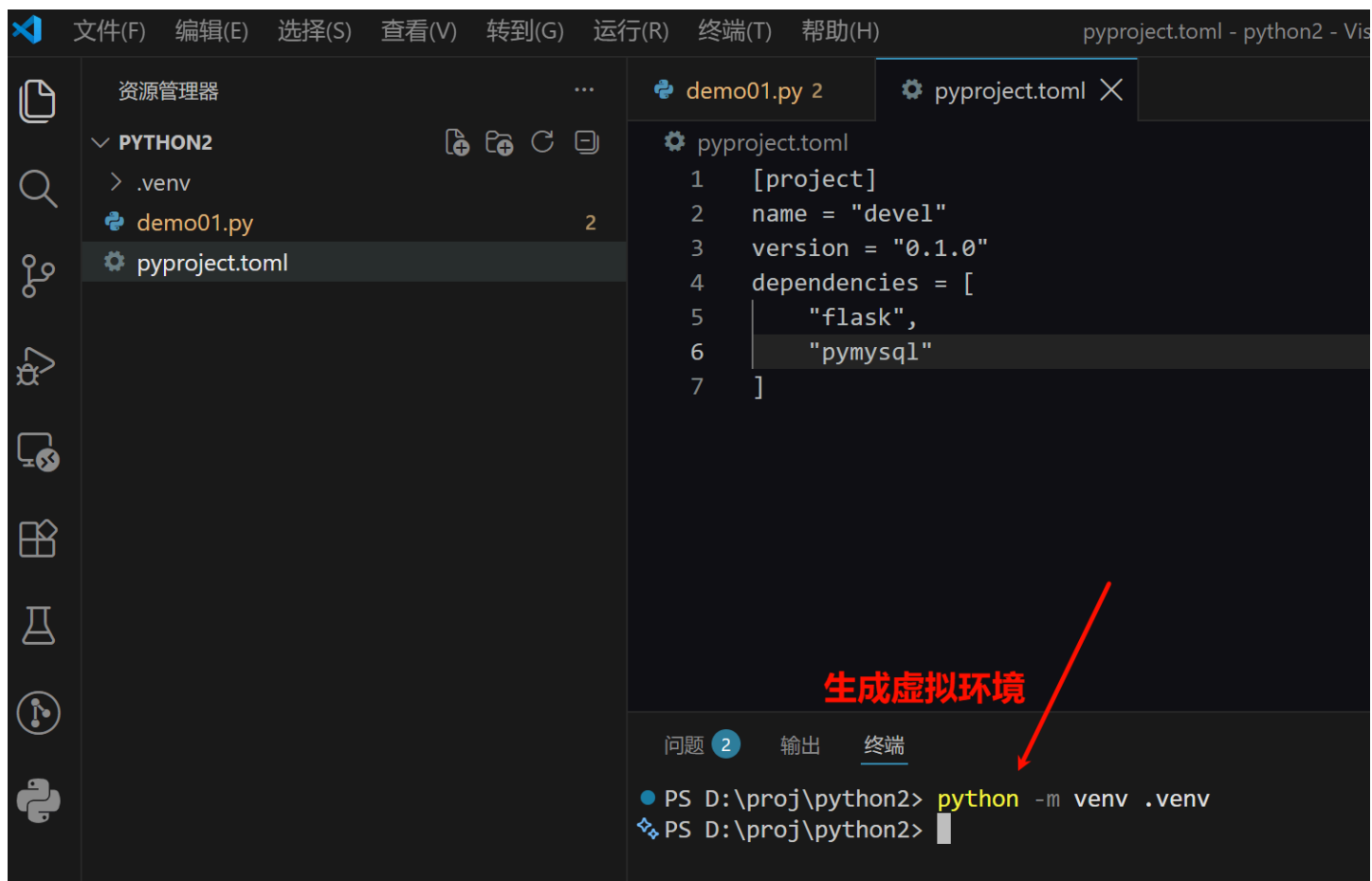
1. 创建 `pyproject.toml` 文件

大概格式如下：

代码块

```
1 [project]
2 name = "project name"
3 version = "0.1.0"
4 dependencies=[
5     "flask",
6     "pymysql"
7 ]
```

2. 测试使用



然后，使用 `pip install -e .` 下载依赖的模块，如果需要增加模块，直接编辑这个文件然后使用这个命令即可触发模块下载。如果这个命令失败了就升级 `pip`。

4.8 UV 的使用

`uv` 是一个Python 包和虚拟环境 管理工具。它的目标是替代 `pip` 和 `virtualenv`，主打 **速度** 和 **可重现性**。总之，它既可以用来下载包又可以管理项目，避免了上述还需要手动编辑项目环境管理文件的情况。

原使用方法:

不灵活，不够自动化，需要手动管理依赖文件。

代码块

```
1 python -m venv .venv # 创建原生虚拟环境，或者使用 conda
2 create pyproject.toml # 创建一个 pyproject.toml 项目管理文件
3 edit pyproject.toml # 将核心依赖写入这个文件
4 pip install -e . # 安装新依赖，每次写入新的依赖到文件中，都需要重新执行这个命令
```

uv的使用方式:

代码块

```
1 uv venv -p 3.9 .venv # 创建虚拟环境
```

```
2 uv init # 初始化, 自动生成相关文件, 包括项目管理文件
3 uv add pk_name # 添加包, 自动处理项目管理文件
4 uv remove pk_name # 删除包, 包的依赖包也会自动卸载
5 uv sync
```

4.8.1 uv使用（结合使用环境使用，按步骤）

- 下载 uv 工具，需要单独下载，官网：<https://docs.astral.sh/uv/> 中有各个平台的下载方式，比如 windows：

代码块

```
1 powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 |
   iex"
```

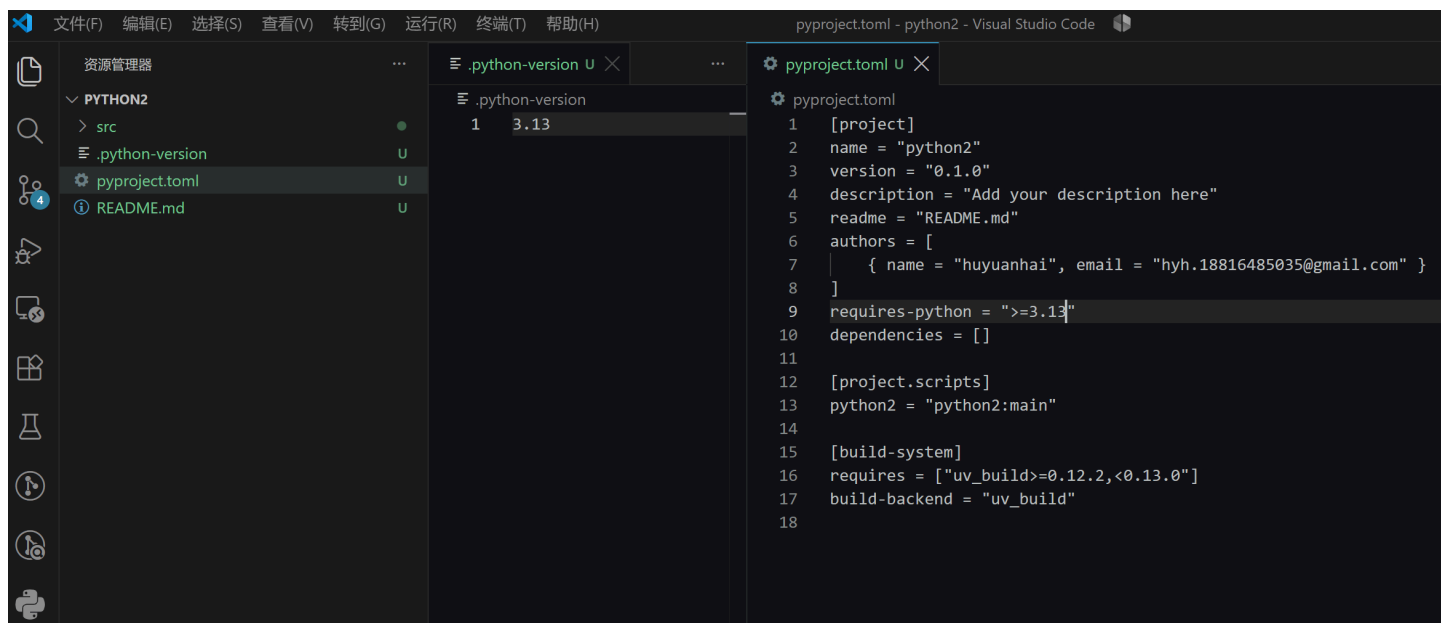
- 初始化工作区

代码块

```
1 # 1、进入一个工程目录, 然后打开终端, 输入:
2 uv init . # 也可以直接指定目录, . 表示当前目录
```

代码块

1	PYTHON2/	# 项目根目录 (即 D:\proj\python2)
2	— src/	# 源码目录 (src-layout 布局)
3	— python2/	# 包目录 (包名即为 python2)
4	— __init__.py	# 包初始化文件, 使 python2 成为可导入的包
5	— .python-version	# 锁定项目使用的 Python 版本 (由 uv 自动生成)
6	— pyproject.toml	# 项目核心配置文件 (声明依赖、元数据、构建系统等)
7	— README.md	# 项目说明文档 (通常用于描述项目用途)

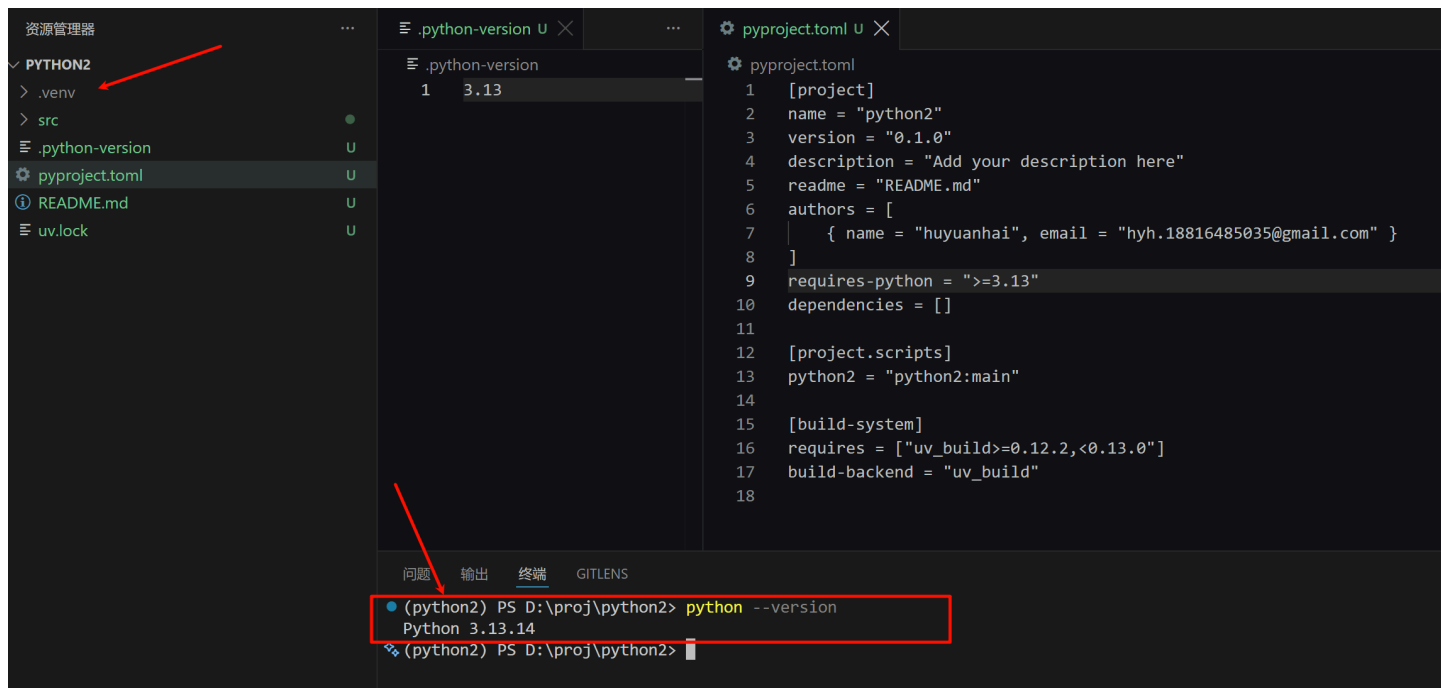


- 环境同步

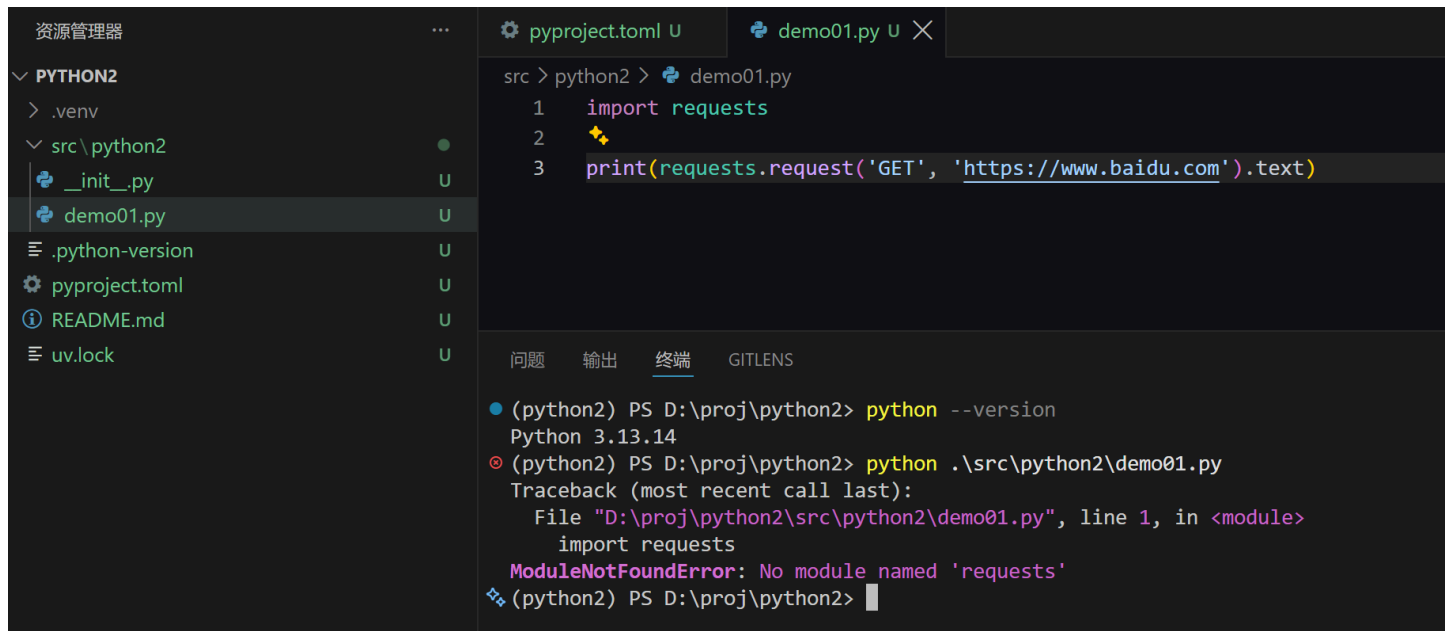
上述只是初始化了工作区，给出了相关的管理文件和项目结构，仍然是没有虚拟环境的。使用 `uv sync`

该命令：根据 `pyproject.toml` 和 `uv.lock` 的描述，把项目的虚拟环境（`.venv` 目录）“调整”到完全一致的状态。

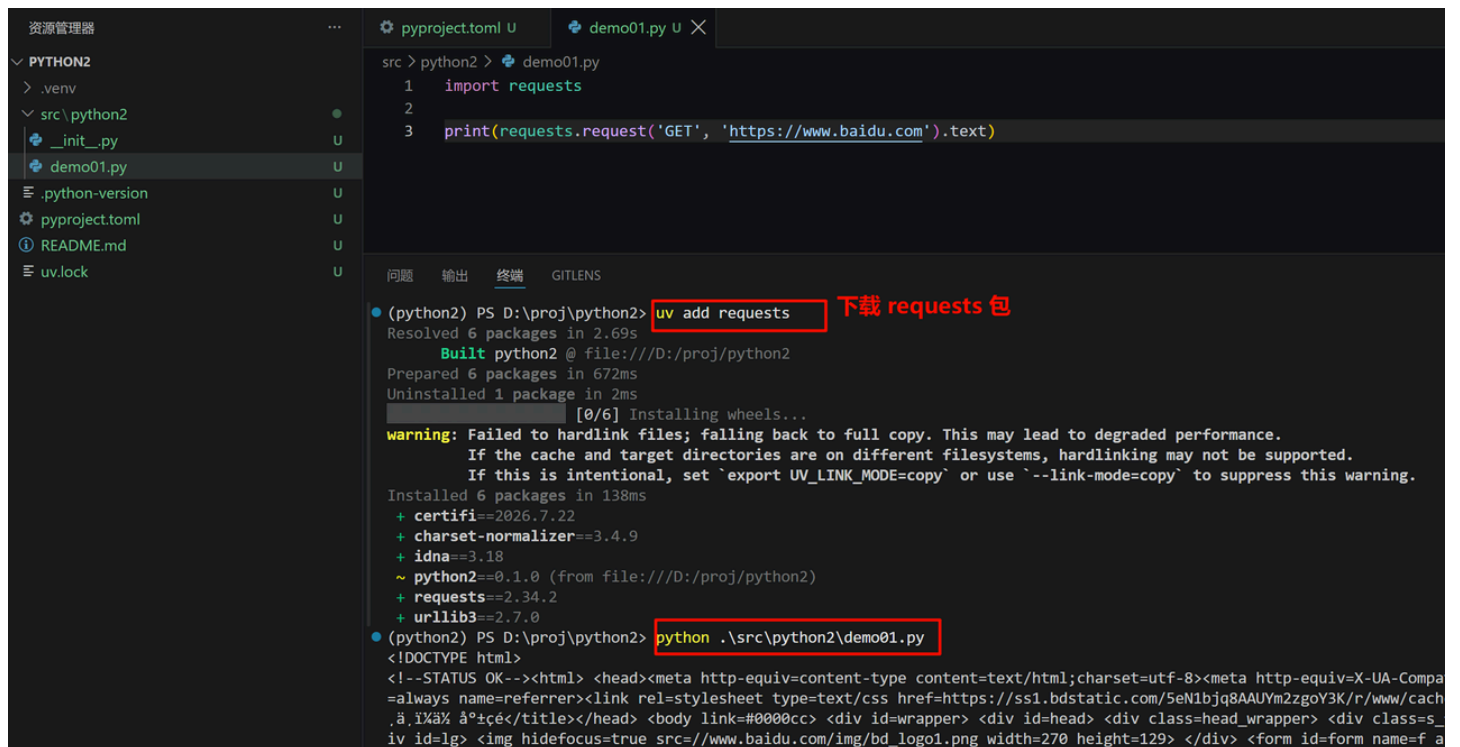
执行后会生成虚拟环境 `.venv` 目录。这里只是下载了对应版本的 python 解释器，还没有第三方依赖。

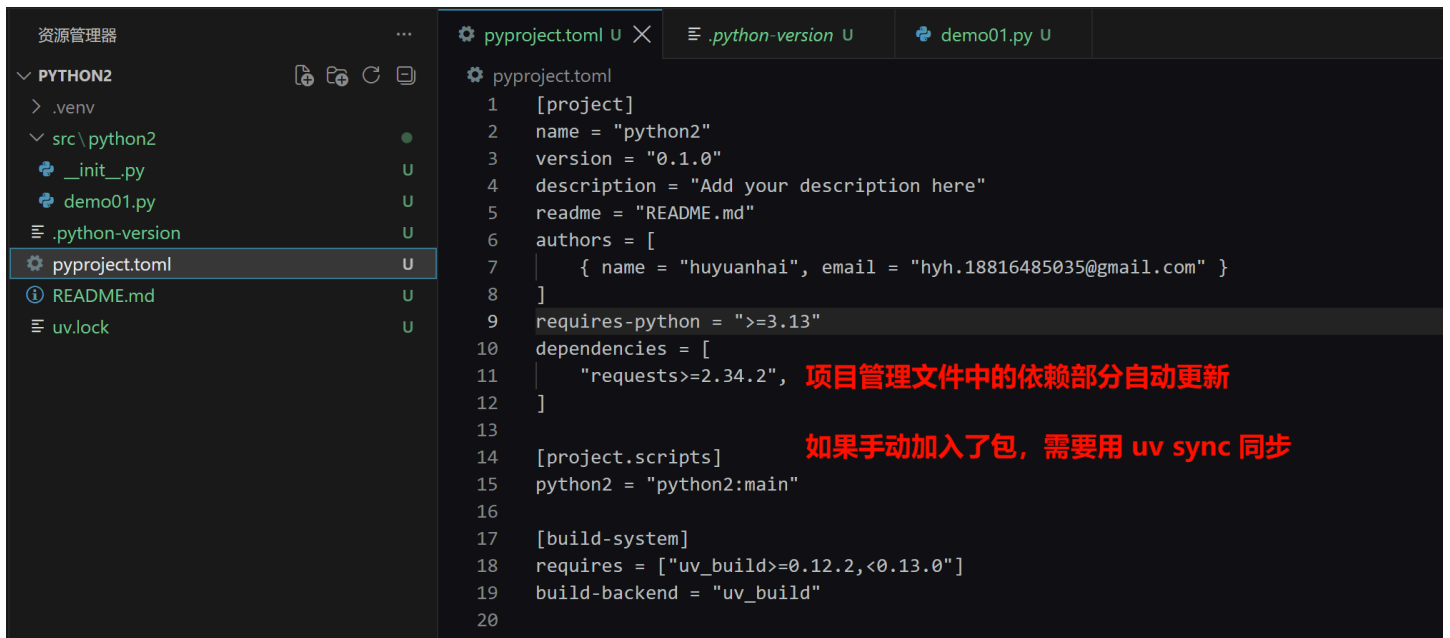


- 编写依赖 `requests` 模块的测试代码文件

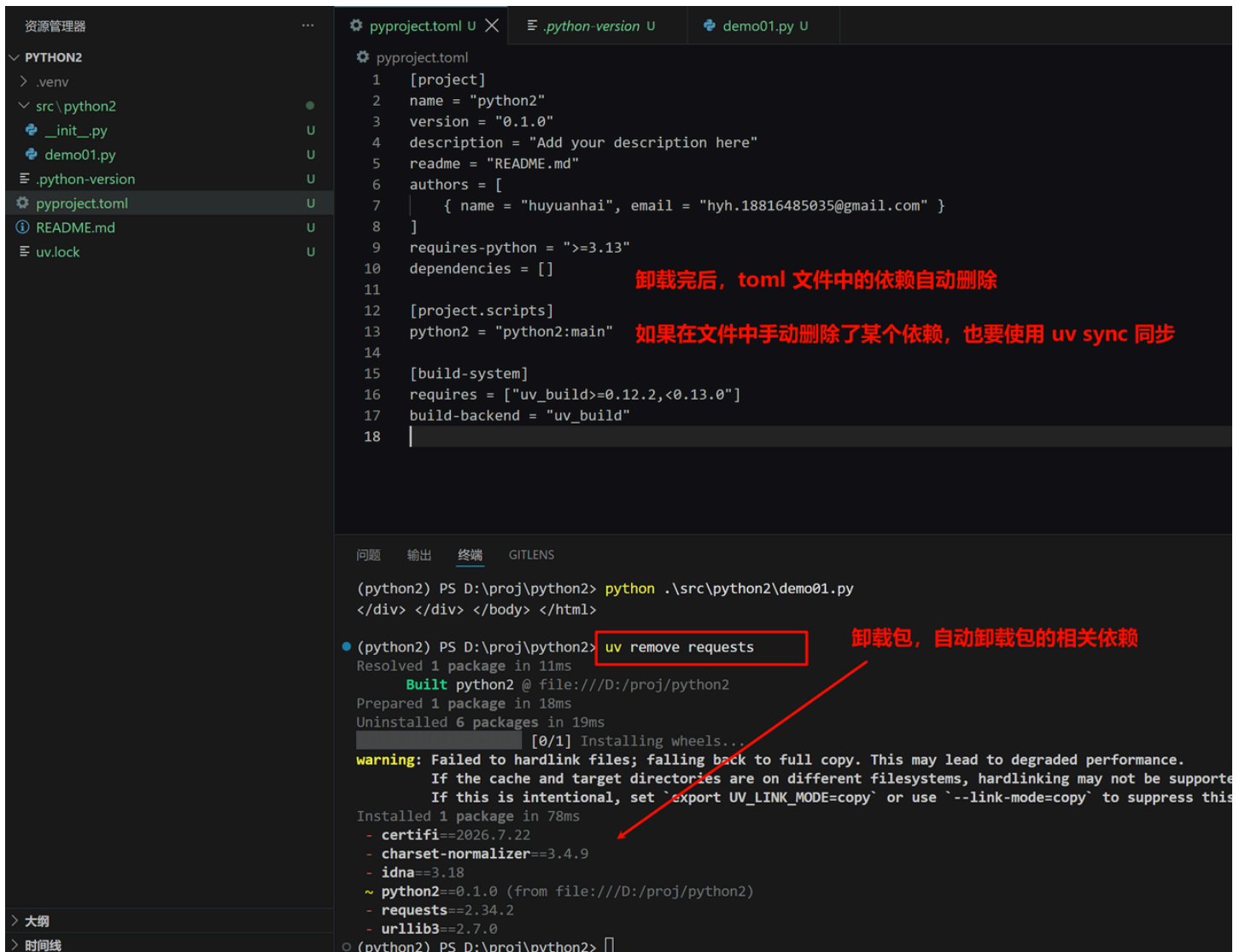


- 使用 `uv add pk_name`





- 删除包 `uv remove pk_name`



4.8.2 命令参考列表

- 基本命令

代码块 1、查看 uv 版本

```
2 (python2) PS D:\proj\python2> uv --version
3 uv 0.12.2 (46ead6098 2026-08-05 x86_64-pc-windows-msvc)
4
5 # 2、查看 uv 帮助信息
6 uv --help
7
8 (python2) PS D:\proj\python2> uv help
9 An extremely fast Python package manager.
10
11 Usage: uv [OPTIONS] <COMMAND>
12
13 Commands:
14   auth                Manage authentication
15   run                 Run a command or script
16   init               Create a new project
17   add                 Add dependencies to the project
18   remove             Remove dependencies from the project
19   version             Read or update the project's version
20   sync               Update the project's environment
21   lock               Update the project's lockfile
22   export              Export the project's lockfile to an alternate
23   format              format
24   tree                Display the project's dependency tree
25   format              Format Python code in the project
26   check              Run checks on the project
27   audit               Audit the project's dependencies
28   tool                Run and install commands provided by Python
29   packages
30   python              Manage Python versions and installations
31   pip                 Manage Python packages with a pip-compatible
32   interface
33   venv                Create a virtual environment
34   build               Build Python packages into source distributions
35   and wheels
36   publish             Upload distributions to an index
37   workspace           Inspect uv workspaces
38   cache               Manage uv's cache
39   self                Manage the uv executable
40   generate-shell-completion Generate shell completion
41   help                Display documentation for a command
42
43 Cache options:
44   -n, --no-cache      Avoid reading from or writing to the cache,
45                       instead using a temporary directory for the duration of the operation [env:
46                       UV_NO_CACHE=]
47   --cache-dir <CACHE_DIR> Path to the cache directory [env: UV_CACHE_DIR=]
```

```

43
44 Python options:
45     --managed-python          Require use of uv-managed Python versions [env:
UV_MANAGED_PYTHON=]
46     --no-managed-python      Disable use of uv-managed Python versions [env:
UV_NO_MANAGED_PYTHON=]
47     --no-python-downloads    Disable automatic downloads of Python. [env:
"UV_PYTHON_DOWNLOADS=never"]
48
49 Global options:
50     -q, --quiet...           Use quiet output
51     -v, --verbose...         Use verbose output
52     --color <COLOR_CHOICE>   Control the use of color in
output [possible values: auto, always, never]
53     --system-certs           Whether to load TLS
certificates from the platform's native certificate store [env:
UV_SYSTEM_CERTS=]
54     --offline                 Disable network access
[env: UV_OFFLINE=]
55     --allow-insecure-host <ALLOW_INSECURE_HOST> Allow insecure connections
to a host [env: UV_INSECURE_HOST=]
56     --no-progress            Hide all progress outputs
[env: UV_NO_PROGRESS=]
57     --directory <DIRECTORY> Change to the given
directory prior to running the command [env: UV_WORKING_DIR=]
58     --project <PROJECT>      Discover a project in the
given directory [env: UV_PROJECT=]
59     --config-file <CONFIG_FILE> The path to a `uv.toml`
file to use for configuration [env: UV_CONFIG_FILE=]
60     --no-config              Avoid discovering
configuration files (`pyproject.toml`, `uv.toml`) [env: UV_NO_CONFIG=]
61     -h, --help               Display the concise help
for this command
62     -V, --version            Display the uv version

```

- Python 环境管理

代码块

```

1  # 1、查看已安装的 python 版本
2  (python2) PS D:\proj\python2> uv python list --only-installed
3  cpython-3.14.0-windows-x86_64-none
C:\Users\huyuanhai\AppData\Local\Programs\Python\Python314\python.exe
4  cpython-3.13.14-windows-x86_64-none
C:\Users\huyuanhai\AppData\Roaming\uv\python\cpython-3.13-windows-x86_64-
none\python.exe

```

```
5 cpython-3.8.3-windows-x86_64-none      C:\Users\huyuanhai\anaconda3\python.exe
6
7 # 2、设置默认 python 版本
8 uv python pin 3.10 # 将当前项目的 Python 版本“固定”为 3.10；修改 .python-version
```

4.8.3 依赖管理

- 添加依赖

代码块

```
1 # 依赖会自动添加到pyproject.toml 文件中，并更新.venv环境。
2 uv add requests
3 uv add "flask>=2.0,<3.0"
```

- 移除依赖

代码块

```
1 # 移除依赖，相关子依赖也会被移除；从 pyproject.toml 文件中删除，更新.venv环境
2 uv remove requests
```

- 更新依赖

代码块

```
1 # 仅针对 requests 这一个包，将其更新到符合项目约束的最新版本，
2 # 并同步更新锁文件（uv.lock），而项目声明文件（pyproject.toml）中的版本约束保持不变。
3 uv lock --upgrade-package requests
4 uv sync
5
6 # 直接升级所有依赖
7 uv lock --upgrade
8 uv sync
```

- 安装项目依赖

代码块

```
1 # 如果
2 uv pip install -e .
```

4.8.4 虚拟环境管理

代码块

```
1  # 创建虚拟环境，生成 .venv
2  uv venv
3
4  # 指定python版本和虚拟环境名称
5  uv venv -p python3.13 my_env
```